COMSATS University Islamabad

Abbottabad, Pakistan

AidLink

By

Naveed Alam CIIT/FA22-BCS-089/ATD

Moazzam Ali CIIT/FA22-BCS-102/ATD

Supervisor

Ms. Sadaf Riaz

Bachelor of Science in Computer Science (2022-2026)

The candidate confirms that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

COMSATS University Islamabad

Abbottabad, Pakistan

AidLink

A project presented to

COMSATS University of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Naveed Alam **CIIT/FA22-BCS-089/ATD**

Moazzam Ali **CIIT/FA22-BCS-102/ATD**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Naveed Alam

Moazzam Ali

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (CS) "AidLink" was developed by **Naveed Alam (CIIT/FA22-BCS-089/ATD)** and **Moazzam Ali (CIIT/FA22-BCS-102/ATD)** under the supervision of Ms. Sadaf Riaz and that in her opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department

(Department of Computer Science)

EXECUTIVE SUMMARY

Access to quality healthcare remains a significant challenge in developing regions, where patients often struggle to locate verified doctors, book appointments in a timely manner, and maintain a continuous record of their medical history. Existing solutions are either fragmented, limited in scope, or fail to address the communication gap between patients and healthcare providers in a unified platform. There is a clear need for a reliable, accessible, and integrated system that can bridge this gap effectively.

To address this problem, AidLink has been developed as a hybrid healthcare application designed to connect patients with verified doctors through a seamless digital platform. The system consists of three interfaces: a Flutter-based mobile application for patients, a Flutter Web portal plus companion mobile application for doctors, and a Flutter Web portal for administrators. All four interfaces are powered by a Firebase backend that handles authentication, real-time data storage, file management, and push notifications.

AidLink enables patients to search for doctors by name, specialty and location, view doctor profiles and ratings, book appointments with real-time slot availability, communicate through an in-app chat system, and access their prescription history at any time. Doctors can manage their schedules, view and respond to appointment requests, issue digital prescriptions, and communicate with patients directly through the platform. The admin portal provides tools for verifying doctor credentials, managing user accounts, moderating reviews, and generating system reports.

The system follows a modular, multi-tiered client-server architecture and is developed using the Iterative and Incremental process model with Object-Oriented Design principles. Firebase Firestore serves as the NoSQL database, Google Maps API powers the location-based doctor discovery feature, and Firebase Cloud Storage handles secure document management. AidLink aims to make healthcare more accessible, transparent, and efficient for both patients and healthcare providers.

ACKNOWLEDGEMENT

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Sadaf Riaz”. Without her personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to her for her encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Naveed Alam

Moazzam Ali

ABBREVIATIONS

SRS	Software Requirements Specification
SDD	Software Design Description
API	Application Programming Interface
UI	User Interface
FCM	Firebase Cloud Messaging
NoSQL	Non-Relational Database
OOD	Object-Oriented Design
FR	Functional Requirement
UC	Use Case
TLS	Transport Layer Security
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
GPS	Global Positioning System
PDF	Portable Document Format
HIPAA	Health Insurance Portability and Accountability Act
AES	Advanced Encryption Standard

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. Introduction

This chapter provides a comprehensive overview of the AidLink project. It begins with a brief description of the system and its purpose, followed by the relevance of the project to the course modules studied during the Bachelor of Science in Computer Science program. The chapter further discusses the background and motivation behind the project, a review of related literature and existing systems, an analysis drawn from that review in the context of AidLink, and the software development methodology and lifecycle model adopted for this project.

1.1 Brief

AidLink is a hybrid healthcare application developed to bridge the communication and accessibility gap between patients and verified doctors. The system is designed as a multi-platform solution consisting of three interfaces: a Flutter-based mobile application for patients targeting Android 8.0 and above as well as iOS 13 and above, a Flutter Web portal for doctors, and a Flutter Web portal for system administrators. All three interfaces are integrated with a Firebase backend that manages user authentication, real-time data storage through Firestore, file and document management through Cloud Storage, serverless logic through Cloud Functions, and push notifications through Firebase Cloud Messaging (FCM).

The outcome of this project is a fully functional healthcare platform that enables patients to discover verified doctors based on specialty and geographic location using the Google Maps API, book appointments with real-time slot availability, communicate with doctors through an end-to-end encrypted in-app chat, and access their prescription history at any time. Doctors are provided with tools to manage their availability schedules, handle appointment requests, issue digital prescriptions, and respond to patients who reach out through the in-app chat for follow-ups and post-consultation queries. The administrator portal offers capabilities for verifying doctor credentials, managing user accounts, moderating patient reviews, and generating system-level reports and analytics.

The project follows an Object-Oriented Design approach and adopts the Iterative and Incremental software development lifecycle, ensuring that the system is developed in manageable phases with continuous refinement at each iteration.

1.2 Relevance to Course Modules

The development of AidLink draws upon knowledge and skills acquired across multiple core and elective courses of the Bachelor of Science in Computer Science program at COMSATS University Islamabad. The project serves as a practical demonstration of how theoretical concepts are applied in the design and development of a real-world software system.

Software Engineering provided the foundation for the entire project lifecycle, from requirements elicitation and specification documented in the Software Requirements Specification (SRS), to system design captured in the Software Design Description

(SDD), and the adoption of the Iterative and Incremental process model to manage development phases systematically.

Object-Oriented Programming formed the basis of the system's design methodology. The entire system is modeled using Object-Oriented Design principles, with key entities such as User, Patient, Doctor, Admin, Appointment, Prescription, ChatMessage, Review, and Notification defined as structured classes with clearly identified attributes and methods.

Database Systems informed the design of the data layer. Firebase Firestore, a NoSQL document-based database, was selected and structured into distinct collections including users, doctors, appointments, prescriptions, chatMessages, reviews, and notifications, with relationships and access patterns designed following core database design principles.

Mobile Application Development provided the practical skills used in building the patient-facing Flutter mobile application, including UI design, state management, and integration with backend services through APIs.

Web Technologies underpinned the development of both the doctor and administrator web portals built with Flutter Web, including layout design, responsive interfaces, and communication with the Firebase backend.

Human-Computer Interaction (HCI) principles guided the design of the user interface, ensuring that core features are accessible within a maximum of two taps on the mobile application and that the appointment booking process is completable within four steps, prioritizing simplicity and usability for all user roles.

1.3 Project Background

Access to quality healthcare remains a pressing challenge in developing regions, including Pakistan, where a significant portion of the population faces difficulties in locating qualified and verified medical professionals. Patients often rely on word-of-mouth referrals, lack access to verified doctor credentials, and have no centralized platform to book appointments, communicate with their doctors, or maintain a continuous digital record of their medical history and prescriptions.

On the other side of the equation, doctors face the challenge of managing their schedules efficiently and handling appointment requests in an organized manner, while patients have no reliable channel to reach their doctors for follow-ups and post-consultation queries outside of physical visits. The absence of a unified digital platform forces both patients and healthcare providers to rely on fragmented, inefficient, and often unreliable methods of interaction.

AidLink was conceptualized to address these challenges by providing a unified, verified, and accessible digital healthcare platform. The system ensures that only credentialed and administrator-approved doctors are visible to patients, thereby establishing a layer of trust and authenticity. By integrating appointment management,

real-time communication, location-based doctor discovery, and prescription management into a single platform, AidLink aims to digitize and streamline the patient-doctor interaction from the point of discovery to post-consultation follow-up. The project is particularly significant in the context of Pakistan's growing smartphone penetration and increasing demand for digital health services, making a mobile-first approach the most practical and impactful design choice.

1.4 Literature Review

The advancement of mobile and cloud technologies has driven significant research in digital healthcare. Ventola [1] highlighted that smartphone-based health tools improve patient engagement and access to healthcare, while emphasizing that user trust and data security are critical factors for adoption. Istepanian et al. [2] established that mobile health systems must prioritize reliability, scalability, and ease of use, principles that directly influenced the design goals of AidLink.

Free et al. [3] found that app-based communication between patients and providers significantly improved appointment adherence and treatment outcomes, reinforcing the need for an integrated in-app chat module in AidLink. Abouelmehdi et al. [4] recommended AES encryption, role-based access control, and secure data transmission as essential requirements for cloud-based healthcare systems, all of which are implemented in AidLink's security architecture.

Existing platforms such as Zocdoc, Practo, and Marham address some aspects of digital healthcare but fall short in key areas. Zocdoc is unavailable in Pakistan and lacks communication features. Practo has an inconsistent doctor verification process. Marham provides doctor listings and appointment booking but lacks real-time in-app chat and integrated prescription management. These limitations provided the primary motivation for developing AidLink as a comprehensive and locally relevant solution. The selection of Flutter as the cross-platform development framework was further validated by Nawrocki et al. [5], who confirmed its suitability for delivering near-native performance across Android, iOS, and Web from a single codebase.

1.5 Analysis from Literature Review

The review of existing literature and platforms reveals that no current solution combines doctor verification, appointment booking, real-time communication, location-based discovery, and prescription management within a single application tailored to the Pakistani healthcare context. Platforms such as Zocdoc, Practo, and Marham address some of these needs individually but fail to deliver a fully integrated experience. AidLink directly addresses this gap by unifying all these features across mobile and web interfaces.

The emphasis on patient trust in the work of Ventola [1] and Abouelmehdi et al. [4] shaped AidLink's administrator-controlled doctor verification workflow and its security architecture, which enforces encrypted communication and role-based access control across all three user interfaces. The findings of Free et al. [3] reinforced the

inclusion of an end-to-end encrypted in-app chat module, ensuring that patient-doctor communication remains private and contained within the platform.

Nawrocki et al. [5] validated Flutter as a cross-platform framework capable of delivering near-native performance across Android, iOS, and Web from a single codebase, making it the most practical choice for a three-interface system developed by a two-person team.

1.6 Methodology and Software Lifecycle for This Project

The development of AidLink followed the Iterative and Incremental software development lifecycle model. Under this model, the system was divided into a series of development iterations, each of which delivered a functional and testable increment of the overall system. The first iteration focused on establishing the authentication module, user registration flows, and the core Firebase backend structure. Subsequent iterations progressively added the doctor verification workflow, appointment booking module, in-app chat, prescription management, location-based doctor discovery, and the administrator reporting and analytics features. Each iteration concluded with a review phase in which the delivered functionality was evaluated against the requirements, and feedback was incorporated into the planning of the next iteration.

The system was designed using Object-Oriented Design principles. All major system entities were identified and modeled as classes with clearly defined attributes, methods, and relationships. The core entities include User, Patient, Doctor, Admin, Appointment, Prescription, ChatMessage, Review, and Notification. These classes form the basis of the class diagram documented in the Software Design Description and are directly reflected in the structure of the Firebase Firestore collections used in the system.

1.6.1 Rationale behind Selected Methodology

The Iterative and Incremental model was selected for AidLink over alternative lifecycle models such as the Waterfall model for several reasons. The system involves three distinct user interfaces and multiple integrated modules, each of which carries a degree of design complexity that benefits from progressive refinement rather than a single, rigid specification-to-delivery cycle. The iterative approach allowed the development team to identify integration issues, refine user interface designs, and adjust module interactions at each iteration, significantly reducing the risk of costly late-stage revisions.

The Waterfall model was specifically excluded because it requires a complete and final specification before any development begins, which is impractical for a multi-platform system where design decisions in one interface often have implications for the others. The iterative model, by contrast, accommodates evolving design insights and allows for continuous validation against the requirements throughout the development process.

Object-Oriented Design was selected as the design methodology because it provides a natural mapping between the real-world entities of the healthcare domain, such as patients, doctors, and appointments, and the software

components that represent them. The encapsulation, inheritance, and modularity principles of OOD ensure that each component of the system is self-contained, independently testable, and replaceable without requiring changes to unrelated parts of the system. This modular structure also supports the future extensibility of AidLink, allowing new features such as video consultations or online payments to be added as separate modules without disrupting the existing architecture.

2. Problem Definition

This chapter defines the core problem that AidLink addresses, outlines the project deliverables and development requirements, and compares the current state of existing healthcare platforms in the context of the identified gaps.

2.1 Problem Statement

Healthcare accessibility in Pakistan is hindered by the absence of a unified digital platform that connects patients with verified doctors. Patients have no reliable way to search for credentialed doctors, book appointments, communicate securely, or maintain a digital record of their prescriptions. Doctors, on the other hand, lack efficient tools to manage their availability and handle appointment requests in an organized manner, while patients have no secure channel to reach their doctors for follow-ups and post-consultation queries. Existing solutions address these needs in isolation, but none provides a fully integrated, locally relevant platform with a verified and trusted doctor network.

AidLink is therefore designed to provide a unified platform that enables patients to discover verified doctors, book appointments with real-time slot availability, communicate through encrypted in-app chat, and access prescription history. Doctors are provided with schedule management, appointment handling, and prescription issuance tools, while administrators oversee doctor verification, user management, and system reporting.

2.2 Deliverables and Development Requirements

This section outlines the expected outputs of the project and the technical prerequisites necessary for their development. The deliverables describe the tangible components and artifacts to be produced, while the development requirements specify the programming environments, libraries, tools, and infrastructure that will enable the successful implementation of the system. Together, these elements ensure that the project is both outcome-driven and technically feasible.

2.2.1 Project Deliverables

The following deliverables are produced as part of the AidLink project:

- A fully functional Flutter mobile application for patients supporting Android 8.0 and above as well as iOS 13 and above.
- A Flutter Web portal for doctors providing schedule management, appointment handling, chat, and prescription tools.
- A Flutter Web portal for administrators providing doctor verification, user management, review moderation, and system reporting.
- A Firebase backend covering authentication, Firestore database, Cloud Storage, Cloud Functions, and FCM notifications.

- Complete project documentation including the SRS, SDD, and this final report.

2.2.2 Development Requirements

The system is built using Flutter as the cross-platform development framework for all three interfaces. Firebase serves as the entire backend, providing Firestore as the NoSQL database, Firebase Authentication for secure login, Firebase Cloud Storage for document management, Firebase Cloud Functions for serverless logic, and Firebase Cloud Messaging for push notifications. Google Maps API is integrated for location-based doctor discovery. The development follows Object-Oriented Design principles and the Iterative and Incremental software development lifecycle model.

2.3 Current System

Several digital healthcare platforms currently operate in relevant markets, each with notable limitations that motivated the development of AidLink.

Zocdoc provides doctor discovery and appointment booking but is restricted to the United States market, making it entirely inaccessible to Pakistani users. It also lacks in-app communication and prescription management features. Practo serves South Asian markets and offers appointment booking and teleconsultation, however its doctor onboarding process is complex, and its verification standards are inconsistent, which undermines patient trust. Marham is the most relevant existing platform in the Pakistani market and supports doctor listings and appointment booking, but it does not provide real-time in-app chat, end-to-end encrypted communication, or integrated prescription generation and management.

None of these platforms offer the combination of administrator-controlled doctor verification, real-time appointment management, encrypted patient-doctor communication, location-based doctor discovery, and digital prescription management within a single unified system. AidLink is developed specifically to fill this gap with a solution that is tailored to the needs and context of the Pakistani healthcare environment.

3. Requirement Analysis

This chapter describes the complete requirements of the AidLink system. It includes the use case diagram for all three user roles, detailed use case descriptions, functional requirements, and non-functional requirements as derived from the Software Requirements Specification (SRS).

3.1 Use Case Diagram

A use case diagram illustrates the interactions between actors and the system. AidLink has three distinct actors: the Patient, the Doctor, and the Admin. Figure 3.1 presents the complete use case diagram of the AidLink system, showing all interactions across all three actor roles within a single unified diagram.

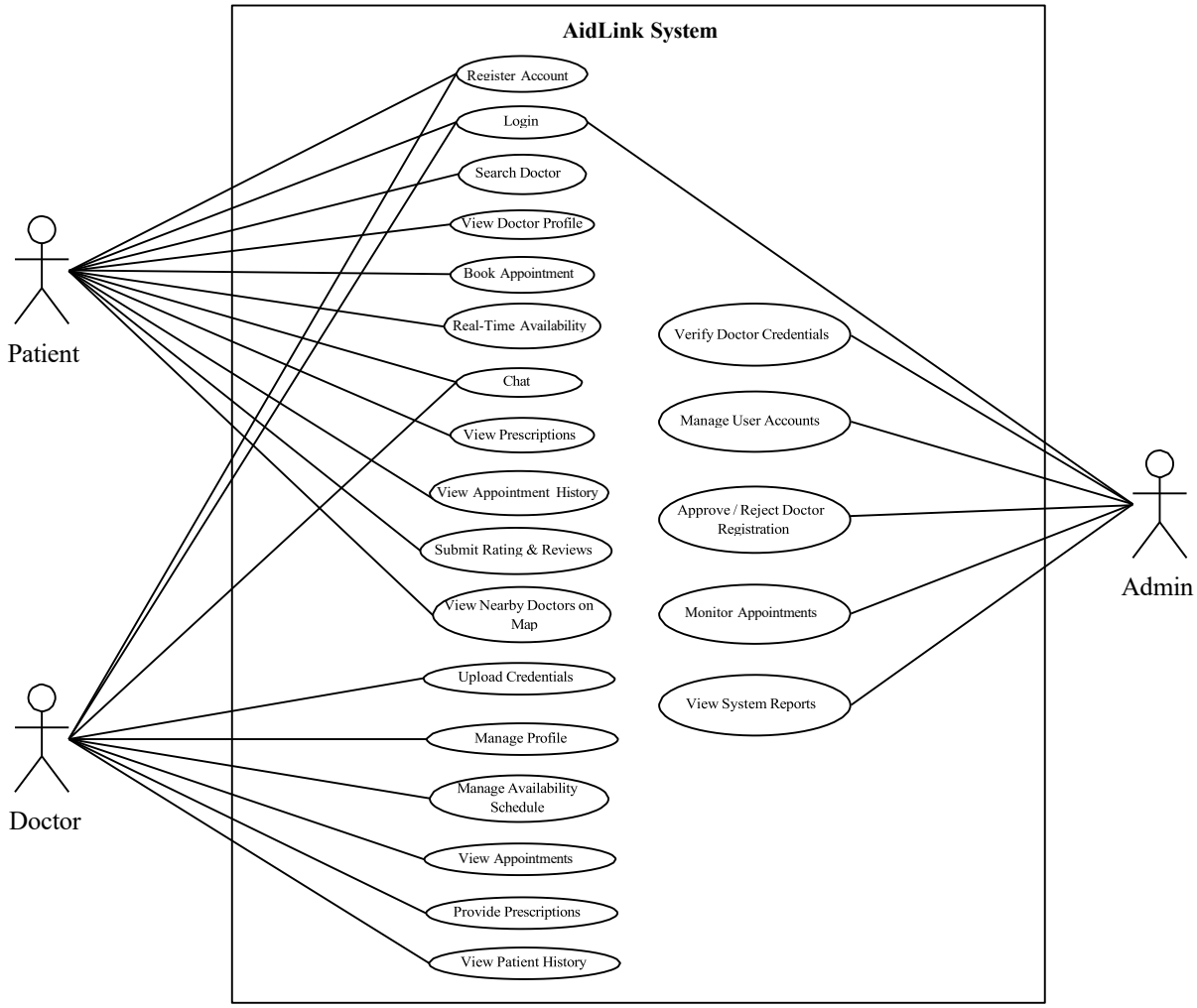

Figure 3.1 Use Case Diagram

3.2 Detailed Use Cases

Detailed use cases provide a comprehensive description of how each actor interacts with the AidLink system to accomplish specific goals.

Table 3.1 presents the detailed use case description for the Registration use case, which allows new patients and doctors to create accounts in the AidLink system.

Table 3.1 Use Case Description for Registration (Patient & Doctor)

Use Case ID	UC-1
Use Case Name	Registration (Patient & Doctor)
Actors	Patient, Doctor
Description	A new user (patient via mobile app or doctor via web portal) creates an account by entering personal details. Doctors additionally upload credentials for verification.
Trigger	User clicks the **"Register"** button on the welcome/login screen.
Preconditions	PRE-1. User is not already registered. PRE-2. Internet connection is available.PRE-3. Registration service is functional.
Postconditions	POST-1. User account is successfully created and stored in the database. POST-2. Confirmation message is displayed. POST-3. Doctor accounts are marked as **Pending Verification**.
Normal Flow	1. User selects "Register." 2. System displays registration form (Patient or Doctor). 3. User enters required details. 4. Doctor uploads credentials (license, degrees). 5. System validates information. 6. System checks uniqueness of email/phone. 7. System creates the user account. 8. System displays success message and redirects to login.
Alternative Flows	3a. Required fields left blank → System prompts user to fill them. 4a. Invalid document format → System asks to upload valid credentials. 5a. Email/phone already exists → System asks user to enter different details.
Exceptions	E1. Internet failure → System shows "Network Error." E2. Server timeout → System displays "Registration Failed. Try Again Later."
Business Rules	BR-1. Password must contain at least 8 characters including a number. BR-2. Email and phone number must be unique. BR-3. Doctor accounts remain inactive until verified by Admin.
Assumptions	A-1. User provides authentic personal information. A-2. Doctor credentials are readable and valid.

Table 3.2 presents the detailed use case description for the Login use case, which authenticates all three user roles and grants them access to their respective dashboards.

Table 3.2 Use Case Description for Login

Use Case ID	UC-2
Use Case Name	Login
Actors	Patient, Doctor, Admin
Description	The user enters valid credentials to access their respective module (mobile app for patient, web portal for doctor/admin). The system authenticates the user and grants access.
Trigger	User clicks the **"Login"** button and enters email and password.
Preconditions	PRE-1. User must already be registered. PRE-2. Internet connection must be available. PRE-3. Authentication service must be operational.
Postconditions	POST-1. User is successfully logged into the system. POST-2. System loads the appropriate dashboard (Patient/Doctor/Admin).
Normal Flow	1. User selects Login. 2. System displays login form. 3. User enters email and password. 4. System validates the input format. 5. System sends credentials for authentication. 6. System verifies user role (Patient/Doctor/Admin). 7. User is redirected to their respective dashboard.
Alternative Flows	3a. User leaves fields blank → System prompts to fill them. 4a. Invalid email format → System shows "Invalid Email Format." 5a. Incorrect password → System displays "Incorrect Password." 5b. Doctor not yet verified → System shows "Your account is pending verification."
Exceptions	E1. Internet connection error → System shows "Network Error." E2. Server timeout → System displays "Login Failed. Try Again Later."
Business Rules	BR-1. Only verified doctors can access the system. BR-2. Admin credentials must match predefined authorized accounts. BR-3. Account will be temporarily locked after 5 failed login attempts.
Assumptions	A-1. User remembers correct credentials. A-2. System clock and server time are synced for session handling.

Table 3.3 presents the detailed use case description for the Search Doctor use case, which allows patients to find verified doctors using filters such as specialization, symptoms, and location.

Table 3.3 Use Case Description for Search Doctor

Use Case ID	UC-3
Use Case Name	Search Doctor
Actors	Patient
Description	The patient searches for a doctor using filters such as specialization, symptoms, or location. The system retrieves and displays a list of matching doctors.
Trigger	Patient clicks the **"Search"** or **"Find Doctors"** option on the home screen.
Preconditions	PRE-1. Patient must be logged in. PRE-2. System must have available doctor data. PRE-3. Internet connection must be active.
Postconditions	POST-1. A filtered list of doctors is displayed. POST-2. Patient may proceed to view doctor profile or book appointment.
Normal Flow	1. Patient selects **Search Doctor**. 2. System displays search screen with filters (specialization, symptoms, location). 3. Patient enters search criteria. 4. System validates input (if needed). 5. System retrieves relevant doctor records from the database. 6. System displays matching doctors in list format.
Alternative Flows	3a. Patient leaves all filters blank → System displays recommended / nearby doctors. 5a. No doctor matches criteria → System displays "No Doctors Found."
Exceptions	E1. Internet connection failure → System shows "Unable to Fetch Results." E2. Server/database not responding → System displays "Service Unavailable. Try Again Later."
Business Rules	BR-1. Only verified doctors should appear in search results. BR-2. Recommendation algorithm prioritizes nearest and highly-rated doctors.
Assumptions	A-1. Patient enters relevant search criteria. A-2. Location services are functional if location-based search is used.

Table 3.4 presents the detailed use case description for the Book Appointment use case, which allows patients to schedule clinic or home visits with verified doctors based on real-time availability

Table 3.4 Use Case Description for Book Appointment

Use Case ID	UC-4
Use Case Name	Book Appointment
Actors	Patient
Description	The patient schedules an appointment with a selected doctor for either a clinic visit or a home consultation based on the doctor's real-time availability.
Trigger	Patient clicks the "Book Appointment" button on the doctor's profile screen.
Preconditions	PRE-1. Patient must be logged in. PRE-2. Doctor must have available time slots. PRE-3. Internet connection must be available. PRE-4. Doctor must be verified by admin.
Postconditions	POST-1. Appointment is created and stored in the system database. POST-2. Doctor receives the appointment notification. POST-3. Patient sees confirmation on screen.
Normal Flow	1. Patient selects "Book Appointment." 2. System displays available dates and time slots. 3. Patient chooses appointment type (clinic visit or home consultation). 4. Patient selects preferred date and time. 5. System validates availability in real-time. 6. System confirms booking and stores appointment details. 7. System displays appointment confirmation message.
Alternative Flows	5a. Selected slot becomes unavailable → System notifies patient and shows updated available slots. 3a. Home visit not offered by doctor → System hides home visit option.
Exceptions	E1. Internet failure → System displays "Connection error. Please try again." E2. Database error → Appointment cannot be saved; system shows "Booking failed."
Business Rules	BR-1. A patient cannot double-book the same doctor at the same time. BR-2. Doctors can define separate schedules for clinic and home visits. BR-3. Appointment fee (if any) must be shown before confirmation.
Assumptions	A-1. Doctor's schedule is updated correctly in the system. A-2. Patient selects a valid date and time.

Table 3.5 presents the detailed use case description for the In-App Chat use case, which allows patients to initiate secure encrypted conversations with doctors for follow-ups and post-consultation queries.

Table 3.5 Use Case Description for In-App Chat

Use Case ID	UC-5
Use Case Name	In-App Chat
Actors	Patient, Doctor
Description	The patient and doctor communicate through a secure in-app chat system for pre-consultation queries, follow-up discussions, and appointment clarifications.
Trigger	Patient or doctor selects the "Chat" option from the appointment details or profile screen.
Preconditions	PRE-1. Patient or doctor must be logged in. PRE-2. A valid appointment must exist between the patient and doctor. PRE-3. Internet connection is required. PRE-4. Chat service must be active.
Postconditions	POST-1. Messages are stored securely in the database. POST-2. Notifications are sent to the recipient. POST-3. Chat history becomes available for future reference.
Normal Flow	1. User selects "Chat." 2. System opens chat window and loads previous messages. 3. User types a message and sends it. 4. System validates and transmits the message. 5. Recipient receives message notification. 6. System updates the chat history in real time.
Alternative Flows	2a. No chat history found → System shows an empty screen with "Start a conversation." 4a. Message contains unsupported media → System shows "File type not supported."
Exceptions	E1. Internet failure → "Message not sent. Try again." E2. Server issue → Chat unavailable temporarily. E3. User blocked → Chat disabled.
Business Rules	BR-1. Chat must be secure and encrypted end-to-end. BR-2. Only patients with confirmed appointments may message doctors. BR-3. Message size and file types must follow system policies.
Assumptions	A-1. Both users respond in a timely manner. A-2. Chat server remains highly available.

Table 3.6 presents the detailed use case description for the Submit Rating and Review use case, which allows patients to provide feedback for doctors after completing an appointment.

Table 3.6 Use Case Description for Submit Rating & Review

Use Case ID	UC-6
Use Case Name	Submit Rating & Review
Actors	Patient
Description	The patient provides a rating and written review for a doctor after completing an appointment, helping other users make informed decisions.
Trigger	Patient selects "Rate Doctor" from the completed appointment details.
Preconditions	PRE-1. Patient must be logged in. PRE-2. Appointment with the doctor must be completed. PRE-3. Review module must be active. PRE-4. Internet connection must be available.
Postconditions	POST-1. Rating and review are saved in the system. POST-2. Doctor's overall rating is updated. POST-3. Review becomes visible to other patients.
Normal Flow	1. Patient opens completed appointment details. 2. Patient selects "Submit Rating & Review." 3. System displays rating scale (1–5 stars) and review text box. 4. Patient selects a star rating and writes a review. 5. Patient submits feedback. 6. System stores rating and review and displays success confirmation.
Alternative Flows	4a. Patient submits rating only → System stores rating without review. 4b. Patient leaves review text empty → System accepts rating alone.
Exceptions	E1. Network error → System shows "Review submission failed." E2. Patient attempts to rate without a completed appointment → System blocks the action. E3. Database failure → Review cannot be stored.
Business Rules	BR-1. Each appointment can receive only one review from the patient. BR-2. Reviews must be moderated by admin for inappropriate content. BR-3. Ratings must be integers between 1 and 5.
Assumptions	A-1. Patient provides honest feedback. A-2. Review moderation occurs promptly.

Table 3.7 presents the detailed use case description for the Upload Credentials use case, which allows doctors to submit their professional documents for administrator verification before accessing the system.

Table 3.7 Use Case Description for Upload Credentials

Use Case ID	UC-7
Use Case Name	Upload Credentials
Actors	Doctor
Description	The doctor uploads required credentials such as medical license, degree certificates, and specialization documents for verification before being allowed to use the system.
Trigger	Doctor selects the "Upload Credentials" option after creating an account or from the profile section.
Preconditions	PRE-1. Doctor must be logged in. PRE-2. Doctor account must be newly created and not yet verified. PRE-3. Required documents must be available in digital format. PRE-4. Internet connection must be available.
Postconditions	POST-1. Credentials are stored in the system database. POST-2. Admin receives a verification request. POST-3. Doctor's status becomes "Pending Verification."
Normal Flow	1. Doctor selects "Upload Credentials." 2. System displays required document list (license, degree, specialization). 3. Doctor uploads the required documents one by one. 4. System validates file format and size. 5. System stores the documents in the database. 6. System notifies the doctor that verification is in progress. 7. Admin receives the verification request.
Alternative Flows	4a. Invalid file type uploaded → System shows "Unsupported format." 4b. File too large → System prompts doctor to compress and re-upload. 3a. Doctor uploads incomplete set of documents → System asks to upload remaining mandatory files.
Exceptions	E1. Internet disconnection during upload → System shows "Upload failed." E2. Server error → System unable to save documents. E3. Doctor cancels upload → No documents are saved.
Business Rules	BR-1. Only verified doctors may provide consultations. BR-2. All mandatory documents must be uploaded before verification begins. BR-3. Admin must manually review and approve credentials.
Assumptions	A-1. Doctor provides genuine and unaltered documents. A-2. System supports all common document formats (JPEG, PNG, PDF).

Table 3.8 presents the detailed use case description for the Manage Availability Schedule use case, which allows verified doctors to set, update, and remove their available time slots for patient bookings.

Table 3.8 Use Case Description for Manage Availability Schedule

Use Case ID	UC-8
Use Case Name	Manage Availability Schedule
Actors	Doctor
Description	The doctor sets, updates, or removes available time slots for clinic visits and home consultations so that patients can book appointments based on accurate real-time schedules.
Trigger	Doctor selects "Manage Availability" from the dashboard.
Preconditions	PRE-1. Doctor must be logged in. PRE-2. Doctor must be verified by admin. PRE-3. Doctor profile must be completed. PRE-4. Internet connection must be available.
Postconditions	POST-1. Updated schedule is saved in the system. POST-2. Patients see updated availability in real time. POST-3. Old slots may be removed or replaced.
Normal Flow	1. Doctor selects "Manage Availability." 2. System displays current schedule with available and booked slots. 3. Doctor selects day/date to edit. 4. Doctor adds, edits, or removes available time slots. 5. System validates changes for conflicting timings. 6. System saves updated schedule. 7. System displays confirmation message.
Alternative Flows	5a. Timing conflict detected → System highlights conflicting slot and asks doctor to adjust. 4a. Doctor removes all slots for a day → System warns "Patients cannot book for this day."
Exceptions	E1. Internet disconnection → System shows "Unable to update schedule." E2. Server failure → Schedule not saved. E3. Doctor tries to modify a slot already booked by a patient → System prevents modification and shows alert.
Business Rules	BR-1. Doctors cannot delete time slots that already have bookings. BR-2. Doctors must specify the appointment type (clinic or home visit) for each slot. BR-3. Real-time updates must be reflected immediately to all users.
Assumptions	A-1. Doctor maintains accurate and updated schedules. A-2. System can sync schedule updates within seconds.

Table 3.9 presents the detailed use case description for the Provide Prescription use case, which allows doctors to issue digital prescriptions for patients following a completed consultation.

Table 3.9 Use Case Description for Provide Prescription

Use Case ID	UC-9
Use Case Name	Provide Prescription
Actors	Doctor
Description	The doctor issues a digital prescription for a patient after a consultation, including medicines, dosage, and doctor notes. The prescription is saved securely in the system for patient access.
Trigger	Doctor selects "Add Prescription" from the appointment details page.
Preconditions	PRE-1. Doctor must be logged in. PRE-2. An appointment must exist between the doctor and patient. PRE-3. Appointment status must be marked as "Completed." PRE-4. Internet connection must be available.
Postconditions	POST-1. Prescription is securely stored in the database. POST-2. Patient receives notification of a new prescription. POST-3. Doctor can view and reference the prescription in future consultations.
Normal Flow	1. Doctor selects "Add Prescription." 2. System displays prescription entry form. 3. Doctor enters medication names, dosage instructions, and notes. 4. Doctor submits the prescription. 5. System validates and saves the prescription. 6. System notifies the patient about the new prescription. 7. System displays confirmation to the doctor.
Alternative Flows	3a. Doctor leaves mandatory fields blank → System prompts to complete them. 4a. Doctor attaches additional files (optional) → System validates supported formats.
Exceptions	E1. Internet failure → System shows "Unable to save prescription." E2. Server/database error → Prescription not stored. E3. Doctor attempts to prescribe without completed appointment → System prevents action.
Business Rules	BR-1. Only verified doctors can issue prescriptions. BR-2. Prescriptions cannot be edited once submitted (only add new ones). BR-3. Prescription must include at least one medication entry.
Assumptions	A-1. Doctor provides accurate medical information. A-2. Patient will follow the prescription instructions.

Table 3.10 presents the detailed use case description for the Verify Doctor Credentials use case, which allows the administrator to review, approve, or reject doctor registration applications based on submitted documents.

Table 3.10 Use Case Description for Verify Doctor Credentials

Use Case ID	UC-10
Use Case Name	Verify Doctor Credentials
Actors	Admin
Description	The admin reviews the documents uploaded by a doctor (license, degree, specialization certificates) and either approves or rejects the doctor's registration based on authenticity and completeness.
Trigger	Admin opens the "Pending Doctor Verifications" list in the admin panel.
Preconditions	PRE-1. Admin must be logged in. PRE-2. Doctor must have uploaded all mandatory credentials. PRE-3. Documents must be accessible in the system database. PRE-4. Internet connection must be available.
Postconditions	POST-1. Doctor account is updated to either "Verified" or "Rejected." POST-2. Notification is sent to the doctor. POST-3. Verification decision is stored in system logs.
Normal Flow	1. Admin selects "Pending Verifications." 2. System displays list of doctors awaiting verification. 3. Admin selects a doctor to review. 4. System displays uploaded documents and doctor details. 5. Admin reviews documents for authenticity. 6. Admin selects either "Approve" or "Reject." 7. System updates doctor's status and notifies the doctor.
Alternative Flows	5a. Documents unclear or incomplete → Admin requests re-upload. 6a. Admin temporarily saves case as "Under Review" to verify manually offline.
Exceptions	E1. Document files fail to load → "Document unavailable." E2. Internet/server issue → Verification cannot be updated. E3. Admin account lacks permission → Access denied.
Business Rules	BR-1. Only admins can verify or reject doctor accounts. BR-2. All required documents must be uploaded before review. BR-3. Verification decisions must be logged for auditing.
Assumptions	A-1. Admin performs verification honestly and professionally. A-2. Uploaded documents are readable and valid.

3.3 Functional Requirements

Functional requirements define the specific behaviours and operations that the AidLink system must support. Each requirement is described individually below with its identifier, title, description, priority, rationale, and associated use case.

Table 3.11 presents the functional requirement for User Registration, which allows new patients and doctors to create accounts in the system.

Table 3.11 FR-001 User Registration

Identifier	**FR-001**
Title	User Registration
Requirement	The system shall allow patients and doctors to register by entering personal details including name, email, password, phone number, and user type.
Source	Stakeholder Interviews
Rationale	Ensures secure onboarding and role-based access.
Business Rule	Password must be at least 8 characters with one number and one special character.
Dependencies	None
Priority	High

Table 3.12 presents the functional requirement for User Login and Logout, which authenticates users and grants role-based access to the system.

Table 3.12 FR-002 User Login & Logout

Identifier	**FR-002**
Title	User Login and Logout
Requirement	The system shall authenticate users and allow secure login and logout using registered credentials.
Source	Functional Scope
Rationale	Provides secure access control and session management.
Business Rule	Account locks after 5 incorrect login attempts.
Dependencies	FR-001
Priority	High

Table 3.13 presents the functional requirement for Doctor Credential Upload, which allows doctors to submit professional documents for verification.

Table 3.13 FR-003 Doctor Credential Upload

Identifier	**FR-003**
Title	Doctor Credentials Upload
Requirement	The system shall allow doctors to upload mandatory credentials including license, degree certificates, and specialization documents.
Source	Admin Verification Requirements
Rationale	Ensures only qualified doctors are approved.
Business Rule	All mandatory documents must be uploaded before verification.
Dependencies	FR-001
Priority	High

Table 3.14 presents the functional requirement for Doctor Verification, which allows the administrator to approve or reject doctor applications.

Table 3.14 FR-004 Doctor Verification

Identifier	**FR-004**
Title	Doctor Verification
Requirement	The system shall allow the admin to review, approve, or reject doctor credentials.
Source	Admin Features
Rationale	Maintains platform trust and authenticity.
Business Rule	Only verified doctors can access appointment and chat features.
Dependencies	FR-003
Priority	High

Table 3.15 presents the functional requirement for Search Doctor, which allows patients to find verified doctors using various filters.

Table 3.15 FR-005 Search Doctor

Identifier	**FR-005**
Title	Search Doctor
Requirement	The system shall allow patients to search doctors by specialization, symptoms, or location.
Source	User Requirements
Rationale	Helps patients find relevant healthcare providers.
Business Rule	Search results must prioritize verified doctors.
Dependencies	FR-002, FR-004

Priority	High

Table 3.16 presents the functional requirement for View Doctor Profile, which allows patients to access detailed information about a doctor before booking.

Table 3.16 FR-006 View Doctor Profile

Identifier	**FR-006**
Title	View Doctor Profile
Requirement	The system shall display detailed doctor profiles including specialization, experience, availability, and ratings.
Source	Functional Scope
Rationale	Enables informed decision-making by patients.
Business Rule	Only verified profiles are publicly visible.
Dependencies	FR-005
Priority	High

Table 3.17 presents the functional requirement for Book Appointment, which allows patients to schedule appointments with verified doctors.

Table 3.17 FR-007 Book Appointment

Identifier	**FR-007**
Title	Appointment Booking
Requirement	The system shall allow patients to book clinic or home visit appointments.
Source	Patient Requirements
Rationale	Enables the core functionality of service scheduling.
Business Rule	Double booking for same time slot must not be allowed.
Dependencies	FR-006
Priority	High

Table 3.18 presents the functional requirement for Real-Time Slot Availability, which ensures patients see accurate and up-to-date scheduling information.

Table 3.18 FR-008 Real-Time Slot Availability

Identifier	**FR-008**
Title	Real-Time Availability
Requirement	The system shall show real-time available appointment slots based on doctor schedules.
Source	System Design
Rationale	Prevents booking conflicts and ensures accurate scheduling.

Business Rule	Updates must reflect schedule changes instantly.
Dependencies	FR-007
Priority	High

Table 3.19 presents the functional requirement for Doctor Schedule Management, which allows doctors to define and update their availability.

Table 3.19 FR-009 Doctor Schedule Management

Identifier	**FR-009**
Title	Doctor Schedule Management
Requirement	The system shall allow doctors to add, update, or remove their availability schedules.
Source	Doctor Requirements
Rationale	Allows doctors to control booking times.
Business Rule	Booked slots cannot be deleted.
Dependencies	FR-008
Priority	High

Table 3.20 presents the functional requirement for In-App Chat, which provides encrypted communication between patients and doctors.

Table 3.20 FR-010 In-App Chat

Identifier	**FR-010**
Title	Patient–Doctor Chat
Requirement	The system shall allow patients and doctors to communicate through secure in-app text messaging.
Source	Communication Needs
Rationale	Enables pre-consultation and follow-up queries.
Business Rule	Only users with an active appointment can chat.
Dependencies	FR-007
Priority	High

Table 3.21 presents the functional requirement for Prescription Creation, which allows doctors to issue digital prescriptions after consultations.

Table 3.21 FR-011 Prescription Creation

Identifier	**FR-011**
Title	Prescription Generation
Requirement	The system shall allow doctors to create and submit digital prescriptions for completed appointments.
Source	Doctor Features
Rationale	Provides proper medical documentation.
Business Rule	Prescriptions cannot be edited once submitted.
Dependencies	FR-007
Priority	High

Table 3.22 presents the functional requirement for View Prescriptions, which allows patients to access and download their prescription history.

Table 3.22 FR-012 View Prescriptions

Identifier	**FR-012**
Title	View Prescriptions
Requirement	The system shall allow patients to view, download, and access their past prescriptions.
Source	Patient Features
Rationale	Helps patients track treatment history.
Business Rule	Only prescriptions linked to patient ID must be shown.
Dependencies	FR-011
Priority	High

Table 3.23 presents the functional requirement for Map-Based Doctor Search, which displays verified doctors on an interactive map.

Table 3.23 FR-013 Map-Based Doctor Search

Identifier	**FR-013**
Title	Map Search
Requirement	The system shall show nearby doctors on a map based on user location.
Source	Location Services
Rationale	Helps patients find easily accessible doctors.
Business Rule	Location permission must be granted by the user.

Dependencies	FR-002
Priority	Medium

Table 3.24 presents the functional requirement for Submit Rating and Review, which allows patients to provide feedback after consultations.

Table 3.24 FR-014 Submit Rating & Review

Identifier	**FR-014**
Title	Ratings & Reviews
Requirement	The system shall allow patients to submit ratings and written reviews for doctors after completed appointments.
Source	User Experience Needs
Rationale	Helps improve transparency and doctor credibility.
Business Rule	One review allowed per appointment.
Dependencies	FR-007
Priority	Medium

Table 3.25 presents the functional requirement for View Appointment History, which maintains a complete record of all past appointments.

Table 3.25 FR-015 View Appointment History

Identifier	**FR-015**
Title	Appointment History
Requirement	The system shall allow patients and doctors to view past and upcoming appointments.
Source	User Needs
Rationale	Provides convenient access to medical and scheduling history.
Business Rule	Appointment records must be immutable.
Dependencies	FR-007
Priority	Medium

Table 3.26 presents the functional requirement for Admin User Management, which allows the administrator to manage all user accounts on the platform.

Table 3.26 FR-016 Admin User Management

Identifier	**FR-016**
Title	Admin User Management
Requirement	The system shall allow the admin to activate, deactivate, or monitor user accounts.

Source	Admin Requirements
Rationale	Ensures compliance and system integrity.
Business Rule	All admin actions must be logged.
Dependencies	FR-002
Priority	High

Table 3.27 presents the functional requirement for Appointment Notifications, which ensures users are informed of booking updates through push notifications.

Table 3.27 FR-017 Appointment Notifications

Identifier	**FR-017**
Title	Notifications
Requirement	The system shall notify users of appointment bookings, cancellations, or updates.
Source	Communication Requirements
Rationale	Keeps users updated on important events.
Business Rule	Notification delivery must occur within 5 seconds.
Dependencies	FR-007
Priority	Medium

Table 3.28 presents the functional requirement for Profile Management, which allows all users to update their personal information.

Table 3.28 FR-018 Profile Management

Identifier	**FR-018**
Title	User Profile Management
Requirement	The system shall allow users to update their personal information including name, contact number, and profile photo.
Source	User Needs
Rationale	Keeps user information accurate.
Business Rule	Email cannot be changed once registered.
Dependencies	FR-001
Priority	Low

Table 3.29 presents the functional requirement for Secure Data Storage, which ensures all sensitive information is encrypted and protected.

Table 3.29 FR-019 Secure Data Storage

Identifier	**FR-019**
Title	Secure Data Storage
Requirement	The system shall securely store all user data, prescriptions, and chat messages using encryption.
Source	Security Requirements
Rationale	Protects sensitive medical information.
Business Rule	System must follow secure data storage standards.
Dependencies	FR-002, FR-011, FR-010
Priority	High

Table 3.30 presents the functional requirement for Role-Based Access Control, which restricts system features based on user roles.

Table 3.30 FR-020 Role-Based Access Control

Identifier	**FR-020**
Title	Access Control
Requirement	The system shall enforce different access permissions for patients, doctors, and admins.
Source	System Architecture
Rationale	Ensures proper authorization and prevents misuse.
Business Rule	Admin has full access; doctors have restricted access; patients only access their own data.
Dependencies	FR-002
Priority	High

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes and constraints that the AidLink system must satisfy. These requirements govern how the system performs rather than what it does, and they are organized into the following categories.

3.4.1 Usability

A new user must be able to learn the basic functions of registration, login, and doctor search within five minutes without external guidance. All core features including Search Doctor, Book Appointment, and Chat must be accessible within two taps from the home screen, and the complete booking process must be completable within four steps.

3.4.2 Performance

A minimum of 95 percent of pages must load within three seconds on a 10 Mbps connection. Real-time slot availability must update within one second, search results must appear within two seconds, and the system must support at least ten thousand concurrent users without degradation.

3.4.3 Security

All sensitive data must be encrypted using AES-256 and all communication must use HTTPS with TLS 1.2 or higher. Accounts must be locked for fifteen minutes after five consecutive failed login attempts, and patient records must comply with HIPAA-like confidentiality standards.

3.4.4 Reliability

The system must maintain a 99 percent success rate for all critical operations including login, booking, and chat. Failed requests must be retried up to three times automatically, and all data must be backed up every 24 hours.

3.4.5 Availability

The system must be available 24 hours a day, seven days a week, with a minimum uptime of 99.5 percent per month. Scheduled maintenance must not exceed three hours per month and users must be notified at least 24 hours in advance.

3.4.6 Maintainability

The codebase must be modular so that any feature can be updated without affecting more than two other modules. All major bugs must be resolved within 72 hours and documentation must be updated after every release.

3.4.7 Portability

The mobile application must support Android 8.0 and above as well as iOS 12 and above. The web portals must run correctly on the latest versions of Chrome, Firefox, Safari, and Edge without additional plugins.

4. Design and Architecture

This chapter presents the complete design specifications of the AidLink system based on the Software Design Description. It covers the design methodology, system architecture, data representation, process flow through activity diagrams, and the core design models including the class diagram and sequence diagrams.

4.1 System Architecture

The architecture of AidLink is based on a modular, multi-tiered client-server design that separates the system into five distinct yet interconnected subsystems: the Patient Module, the Doctor Module, the Admin Module, the Backend Services Module, and the Map and Location Services Module. This modular approach ensures clean separation of concerns, high maintainability, and the ability to scale the system as new features are introduced.

All client applications, whether mobile or web, communicate with a centralized backend hosted on Firebase through secure API calls. The patient module operates as a Flutter mobile application, while both the doctor and admin dashboards are implemented using Flutter Web, ensuring a unified codebase and consistent design language across all three interfaces. The system integrates the Google Maps API to support location-aware services such as identifying nearby doctors and navigating to clinics.

The five major subsystems of AidLink are described below. The Patient Module is a Flutter mobile application that provides features including registration, login, doctor search, profile viewing, appointment booking, real-time availability checking, in-app chat, prescription viewing, and map-based doctor discovery. The Doctor Module is a Flutter Web application that enables doctors to manage profiles, upload credentials, set availability schedules, view and manage appointments, chat with patients, and issue electronic prescriptions. The Admin Module is a Flutter Web application that allows administrators to verify doctor credentials, approve or reject registrations, manage user accounts, moderate reviews, and generate system-level reports. The Backend Services Module is powered by Firebase and handles user authentication, real-time data management through Firestore, file storage for documents and prescriptions through Cloud Storage, and serverless logic through Cloud Functions. The Map and Location Services Module integrates the Google Maps API to provide geolocation features, nearby doctor search, and clinic navigation support.

Figure 4.1 presents the high-level architecture diagram of the AidLink system.

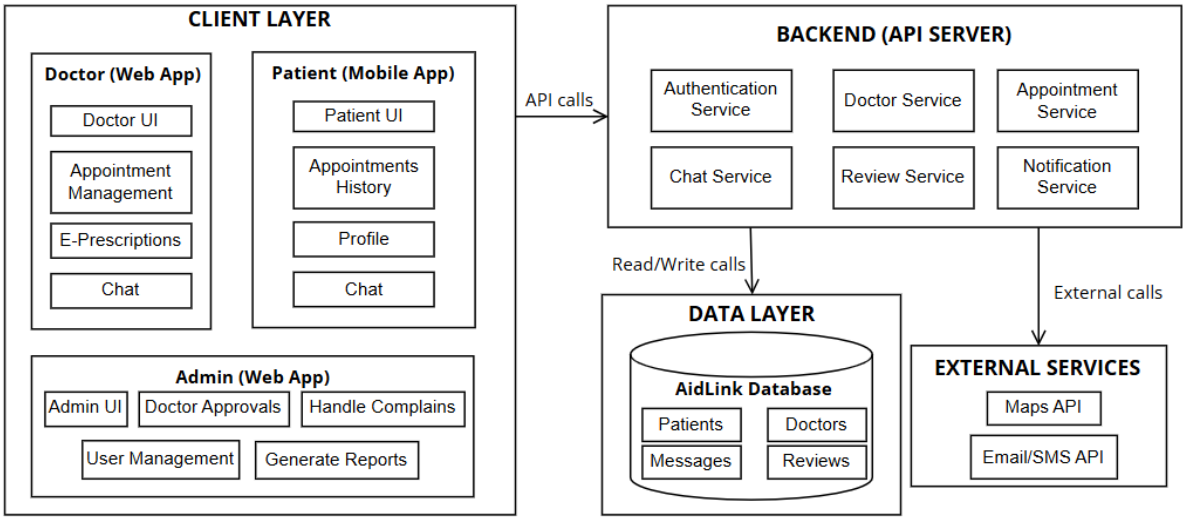

Figure 4.1 High-Level Architecture Diagram

4.2 Data Representation

The information domain of AidLink is modeled using an object-oriented approach where core entities such as Users, Patients, Doctors, Appointments, Prescriptions, Chat Messages, Reviews, and Notifications are represented as classes in the system. These entities are mapped directly to Firebase Firestore, which stores data using a NoSQL document-based structure organized into collections and documents. Relationships between entities are handled through document references such as userId, doctorId, and appointmentId, allowing flexible linking without requiring joins. Real-time synchronization is enforced through Firestore listeners, and security is implemented through Firebase Authentication and Firestore Security Rules, ensuring that each user role only accesses permitted data. Firebase Cloud Storage is used for storing doctor credential files and prescription PDF documents.

Table 4.1 presents the complete data dictionary for the AidLink system, listing all entities with their attributes, data types, and descriptions.

Table 4.1 Data Dictionary

Entity	Attribute / Method	Type	Description
User	userId	String	Unique Firebase Auth UID
User	name	String	Full name of the user
User	email	String	Login email
User	password	String	Encrypted password
User	phone	String	Registered phone number
User	role	Enum	Patient / Doctor / Admin
User	status	String	Active / Pending / Blocked

User	login()	Method	Validates Firebase Auth credentials
User	logout()	Method	Ends user session
User	updateProfile()	Method	Updates user information
Patient	age	Int	Patient age
Patient	gender	String	Male / Female / Other
Patient	address	String	Residential address
Patient	bookAppointment()	Method	Books a new appointment
Patient	viewPrescription()	Method	Retrieves prescriptions
Patient	sendMessage()	Method	Sends chat message to doctor
Patient	writeReview()	Method	Submits review after consultation
Doctor	doctorId	String	Unique identifier
Doctor	specialization	String	Medical specialization
Doctor	experienceYears	Int	Years of experience
Doctor	clinicAddress	String	Clinic location
Doctor	consultationFee	Double	Consultation charges
Doctor	credentials	String (URL)	Uploaded certificate paths
Doctor	approvalStatus	String	Pending / Approved / Rejected
Doctor	approveAppointment()	Method	Confirms patient booking
Doctor	createPrescription()	Method	Issues medicine prescription
Doctor	sendMessage()	Method	Chats with patient
Doctor	updateSchedule()	Method	Modifies availability slots
Appointment	appointmentId	String	Unique Firestore document ID
Appointment	patientId	String	Reference to Patient
Appointment	doctorId	String	Reference to Doctor
Appointment	date	Date	Appointment date
Appointment	time	String	Time slot
Appointment	status	String	Pending / Confirmed / Cancelled
Appointment	locationType	String	Clinic Visit / Home Visit
Appointment	confirm()	Method	Marks appointment as confirmed

Appointment	cancel()	Method	Cancels appointment
Prescription	prescriptionId	String	Auto-generated ID
Prescription	appointmentId	String	Linked appointment
Prescription	doctorNotes	String	Medical notes
Prescription	medicineList	String	Prescribed medicines
Prescription	generatedDate	Date	Date of creation
Prescription	generatePDF()	Method	Exports prescription to PDF, uploads to Cloud Storage, and saves download link to Firestore
ChatMessage	messageId	String	Auto-generated Firestore ID
ChatMessage	senderId	String	Reference to sending user
ChatMessage	receiverId	String	Reference to receiving user
ChatMessage	messageText	String	Text message content
ChatMessage	timestamp	DateTime	Time the message was sent
ChatMessage	messageType	String	Text / Image / File
ChatMessage	send()	Method	Sends message in real time
Review	reviewId	String	Unique review ID
Review	patientId	String	Reviewer (patient)
Review	doctorId	String	Doctor being reviewed
Review	rating	Int	Rating from 1 to 5
Review	comment	String	Review text
Review	date	Date	Submission date
Review	status	String	Visible / Hidden
Review	submitReview()	Method	Adds review to database
Notification	notificationId	String	Unique notification ID
Notification	userId	String	Notification receiver
Notification	message	String	Notification message
Notification	timestamp	DateTime	Time notification was sent
Notification	isRead	Boolean	Read/unread indicator
Notification	send()	Method	Sends push notification via FCM

Notification	markAsRead()	Method	Updates read status to true

4.3 Process Flow/Representation

The process flow of the AidLink system is illustrated through three activity diagrams, one for each user role. Each diagram captures the complete sequence of actions available to that actor from login through to logout, including all parallel activities and decision points.

4.3.1 Patient Activity Diagram

Figure 4.2 illustrates the sequence of actions a patient performs within the AidLink mobile application. After a successful login, the patient is directed to the dashboard from where multiple activities can occur in parallel, including opening the chat module, viewing prescriptions, checking appointment history, and viewing nearby doctors on the map. The diagram also captures the appointment booking process, where the patient searches for a doctor, views available profiles, and if slots are available, selects the visit type and confirms the booking. The activity concludes when the patient logs out.

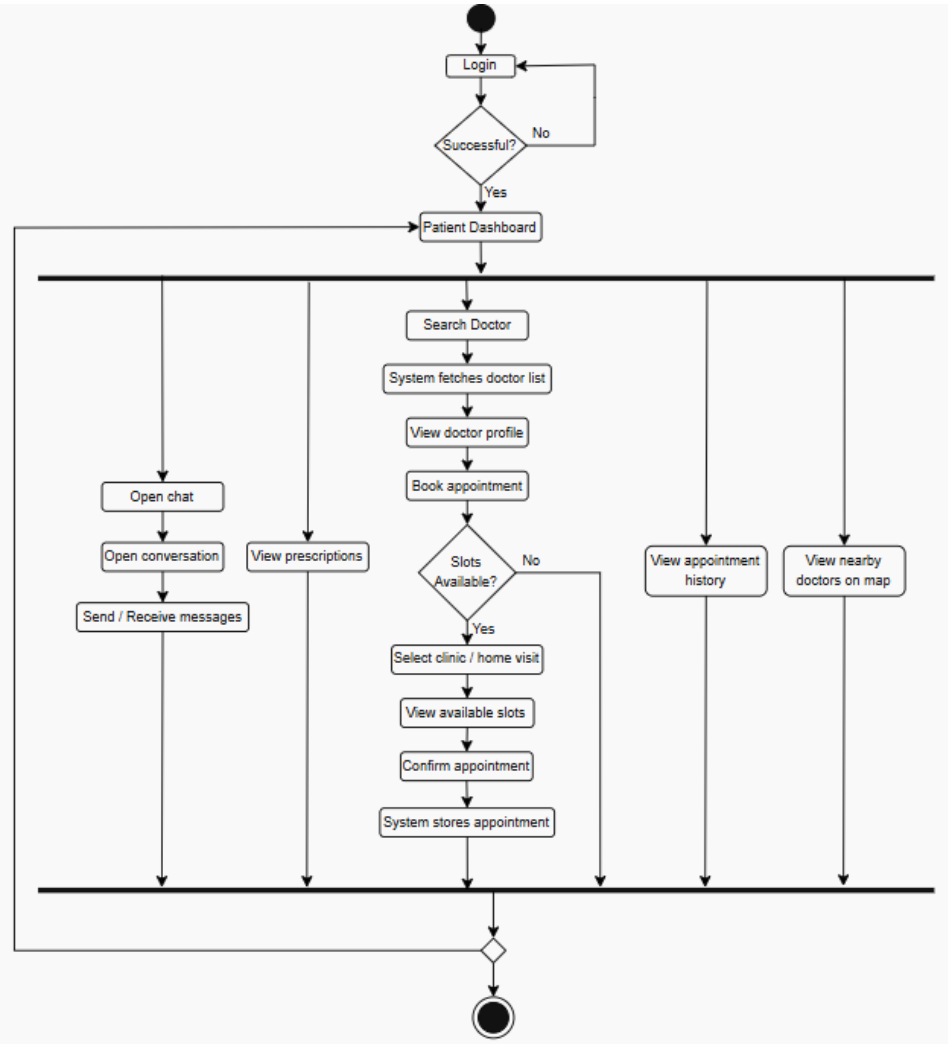

Figure 4.2 Patient Activity Diagram

4.3.2 Doctor Activity Diagram

Figure 4.3 represents the sequence of actions performed by the doctor within the AidLink web portal. After logging in, the doctor is taken to the dashboard from where multiple operations can occur in parallel. The doctor can view appointments, open a selected appointment, review patient details, conduct the consultation, communicate via chat, issue a prescription, and update the appointment status. Simultaneously, the doctor can manage their availability schedule, access patient history, and update their profile. All activities converge back to the main dashboard, and the session ends when the doctor logs out.

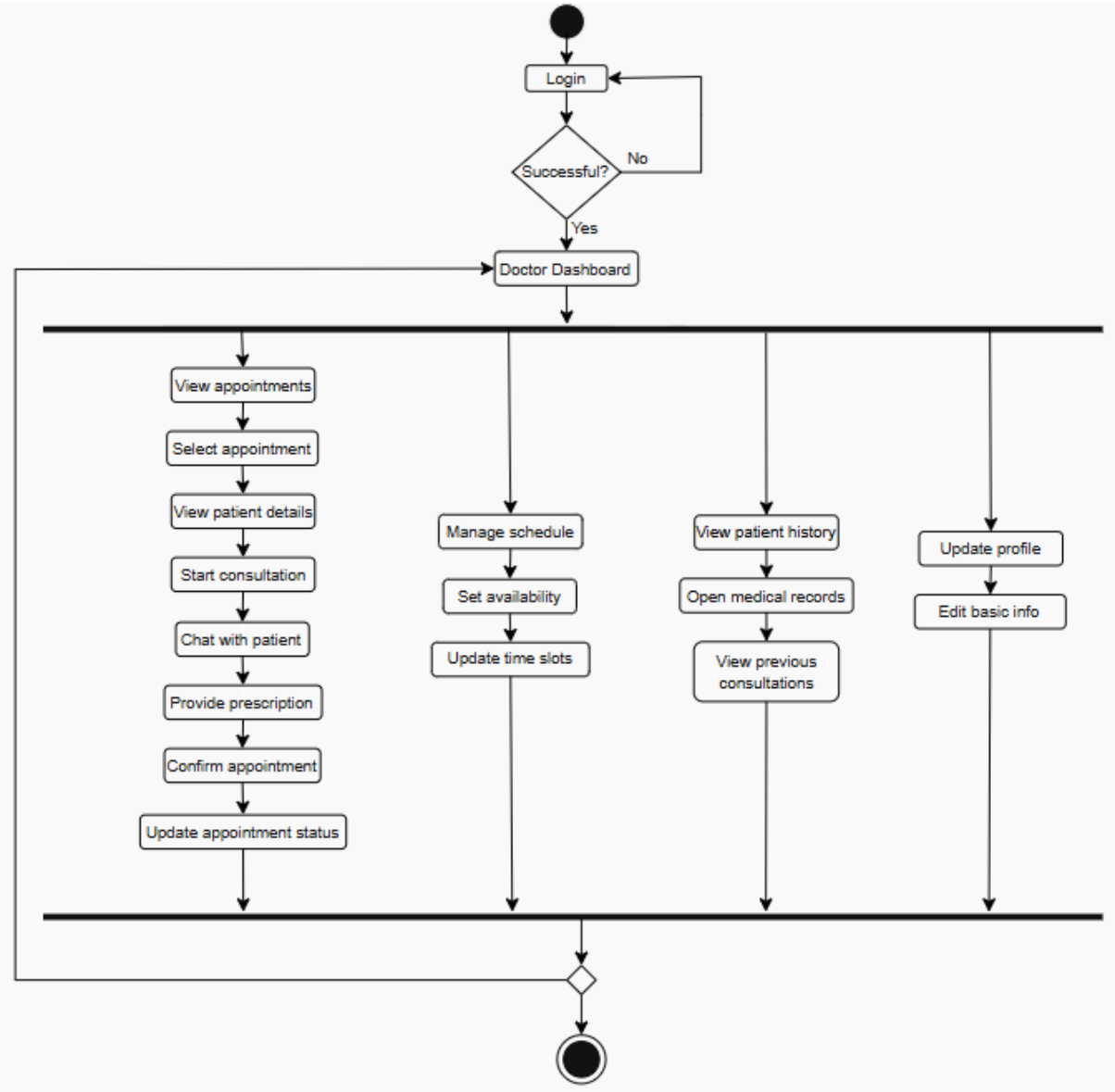

Figure 4.3 Doctor Activity Diagram

4.3.3 Admin Activity Diagram

Figure 4.4 illustrates how the system administrator interacts with the AidLink web portal. After login, the admin is directed to the dashboard from where multiple administrative tasks can be performed in parallel. These include reviewing pending doctor registration requests, verifying submitted documents, and approving or rejecting applications. The admin can also view analytical reports, generate monthly summaries, manage user accounts, moderate patient reviews, and update system-level settings. All parallel activities return to the admin dashboard and the workflow concludes when the admin logs out.

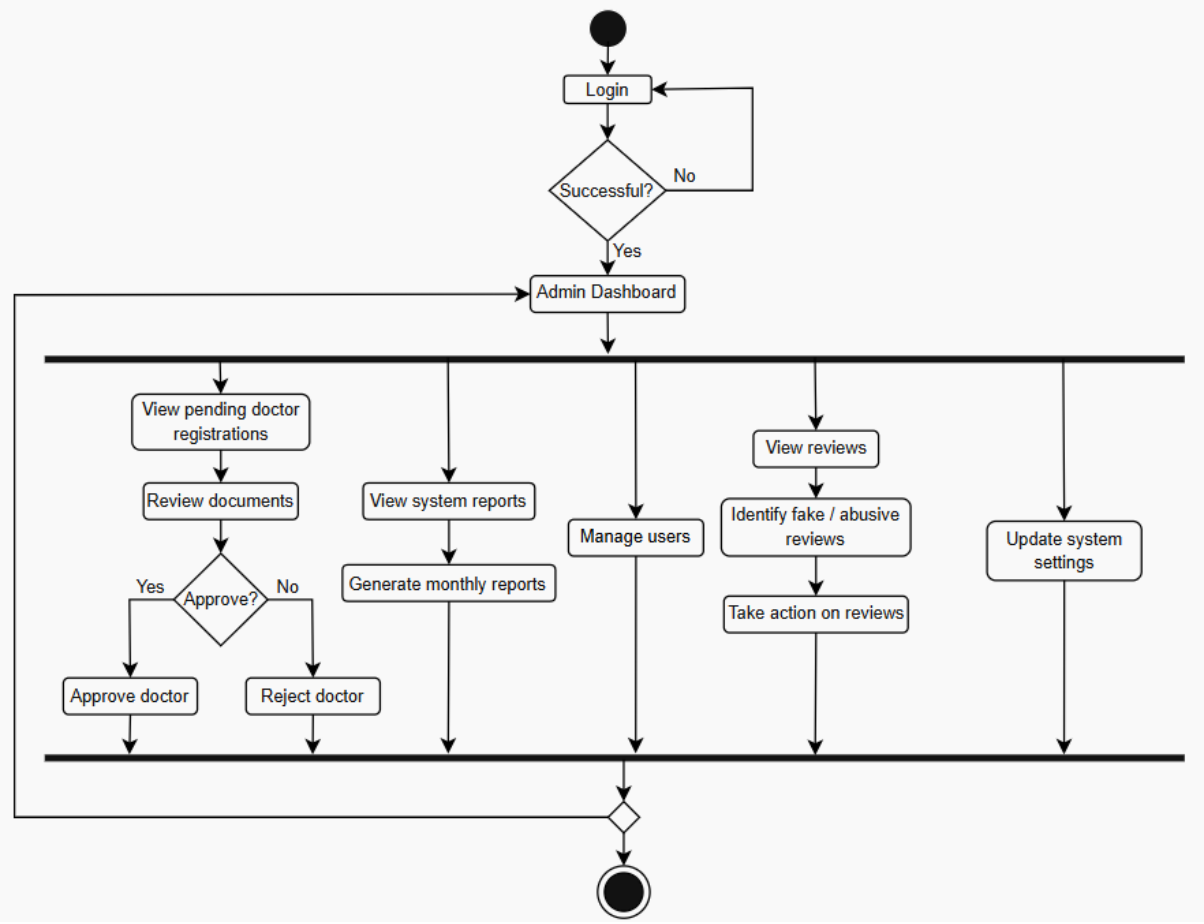

Figure 4.4 Admin Activity Diagram

4.4 Design Models

The AidLink system is designed using Object-Oriented Design principles. The design models include a class diagram that defines the structural foundation of the system and three sequence diagrams that illustrate the runtime interaction flows for each user role.

4.4.1 Class Diagram

The AidLink class diagram represents the core structural model of the system by defining its key entities, their attributes, primary operations, and the associations between them. At the top level, a general User class encapsulates common information such as name, email, phone, login credentials, and system role. The system extends this into three specialized types: Patient, Doctor, and Admin, each with additional attributes and behaviors suited to their responsibilities.

Supporting classes including Appointment, Prescription, ChatMessage, Review, and Notification capture operational data and system-generated interactions. Appointments link patients and doctors, prescriptions are generated after consultations, and chat messages allow real-time communication. Reviews enable patients to rate doctors, while notifications ensure users receive important updates. Figure 4.5 presents the complete class diagram of the AidLink system.

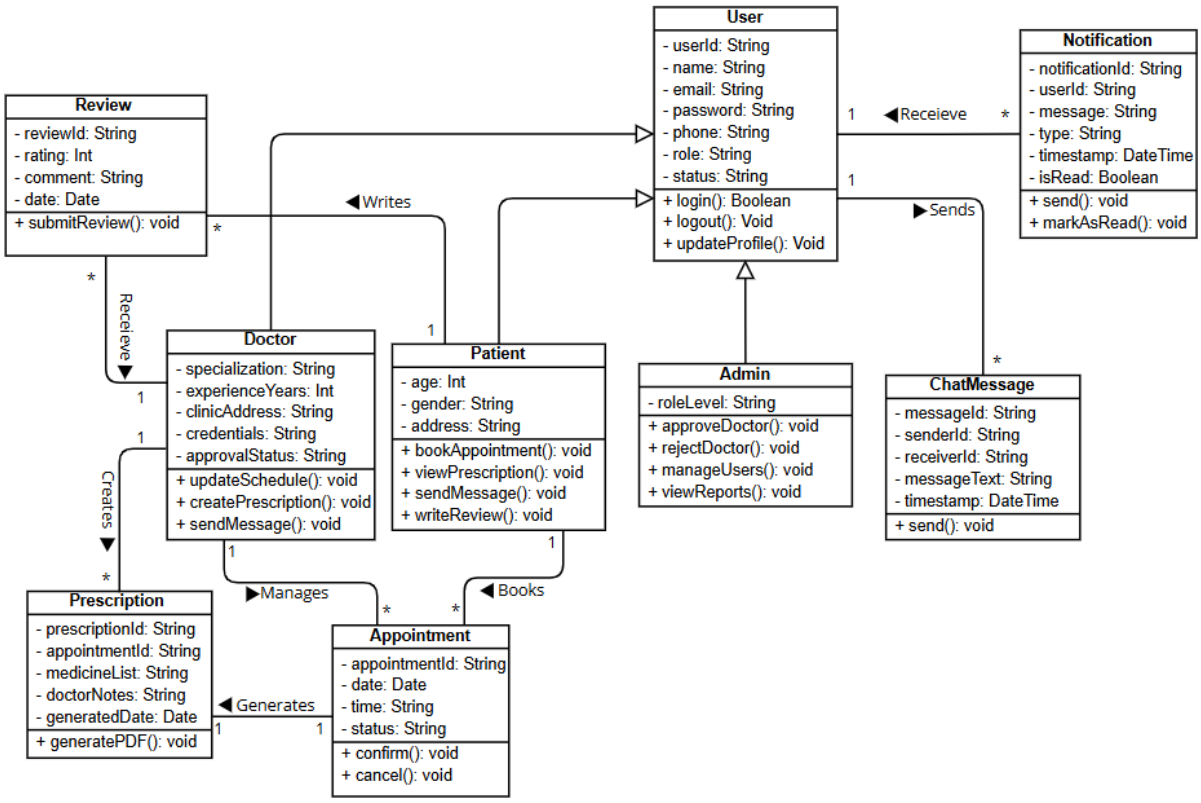

Figure 4.5 AidLink Class Diagram

4.4.2 Sequence Diagrams

The sequence diagrams illustrate the runtime interaction flows between actors, application modules, and the Firebase backend for each of the three user roles.

Figure 4.6 presents the Patient Sequence Diagram, which illustrates the interaction flow between the patient, the mobile application, and the backend throughout key operations. The sequence begins with login or registration, after which the patient accesses the dashboard and can search for doctors, view profiles, book appointments, view prescriptions, access appointment history, and send or receive chat messages. When the patient books an appointment, the request is stored by the backend and forwarded to the corresponding doctor. Once the doctor responds, the backend updates the status and notifies the patient.

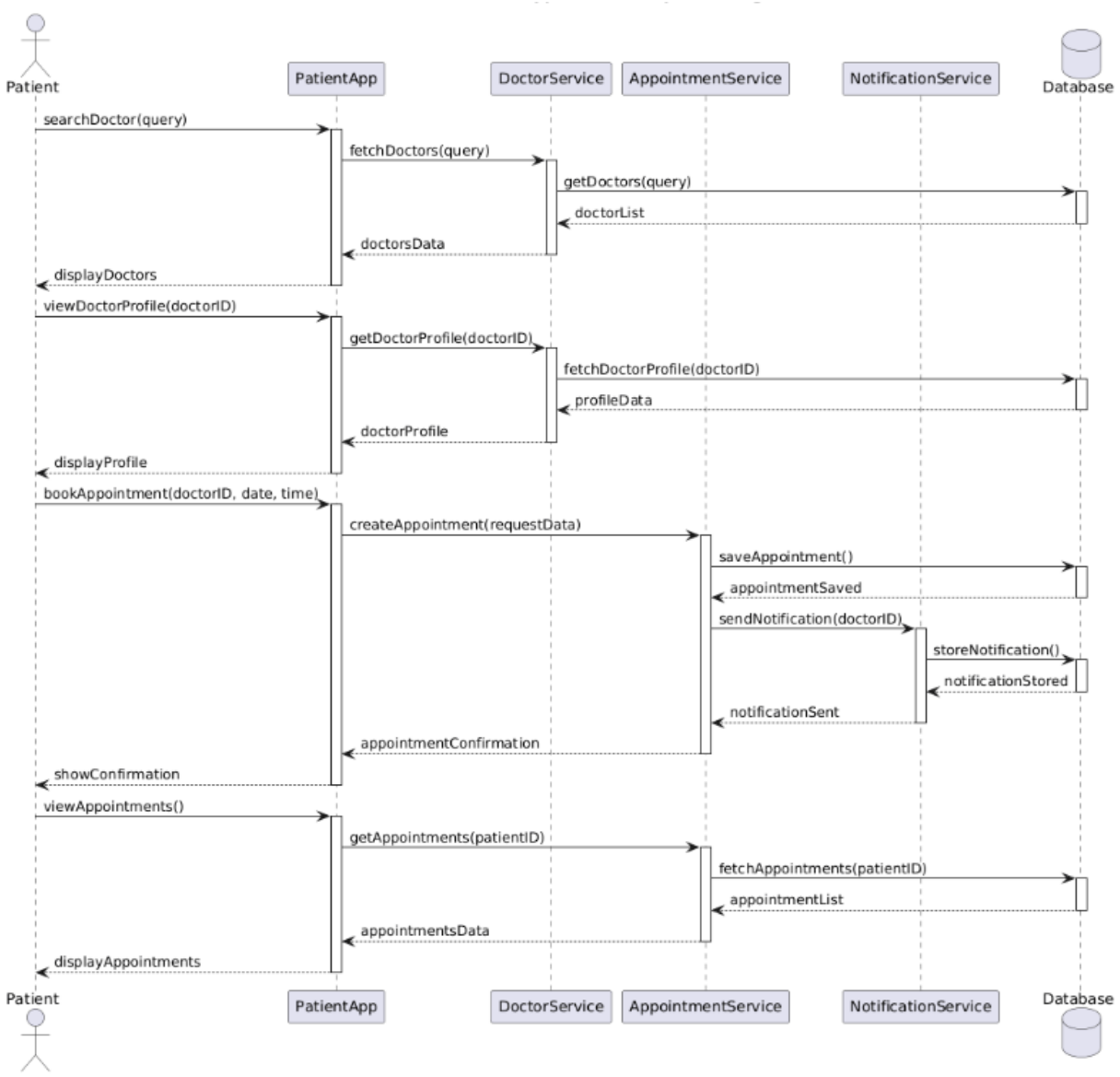

Figure 4.6 Patient Sequence Diagram

Figure 4.7 presents the Doctor Sequence Diagram, which illustrates the interaction flow when a doctor manages patient consultations. The doctor begins by retrieving the list of assigned appointments from the Appointment Module. When a specific appointment is selected, patient information is fetched from the Patient Records component. If the doctor initiates communication, the Chat Module loads the previous conversation history. During or after the consultation, the doctor generates a prescription through the Prescription Module, which stores the data and confirms successful completion. Finally, the doctor updates the appointment status, which is saved to the database.

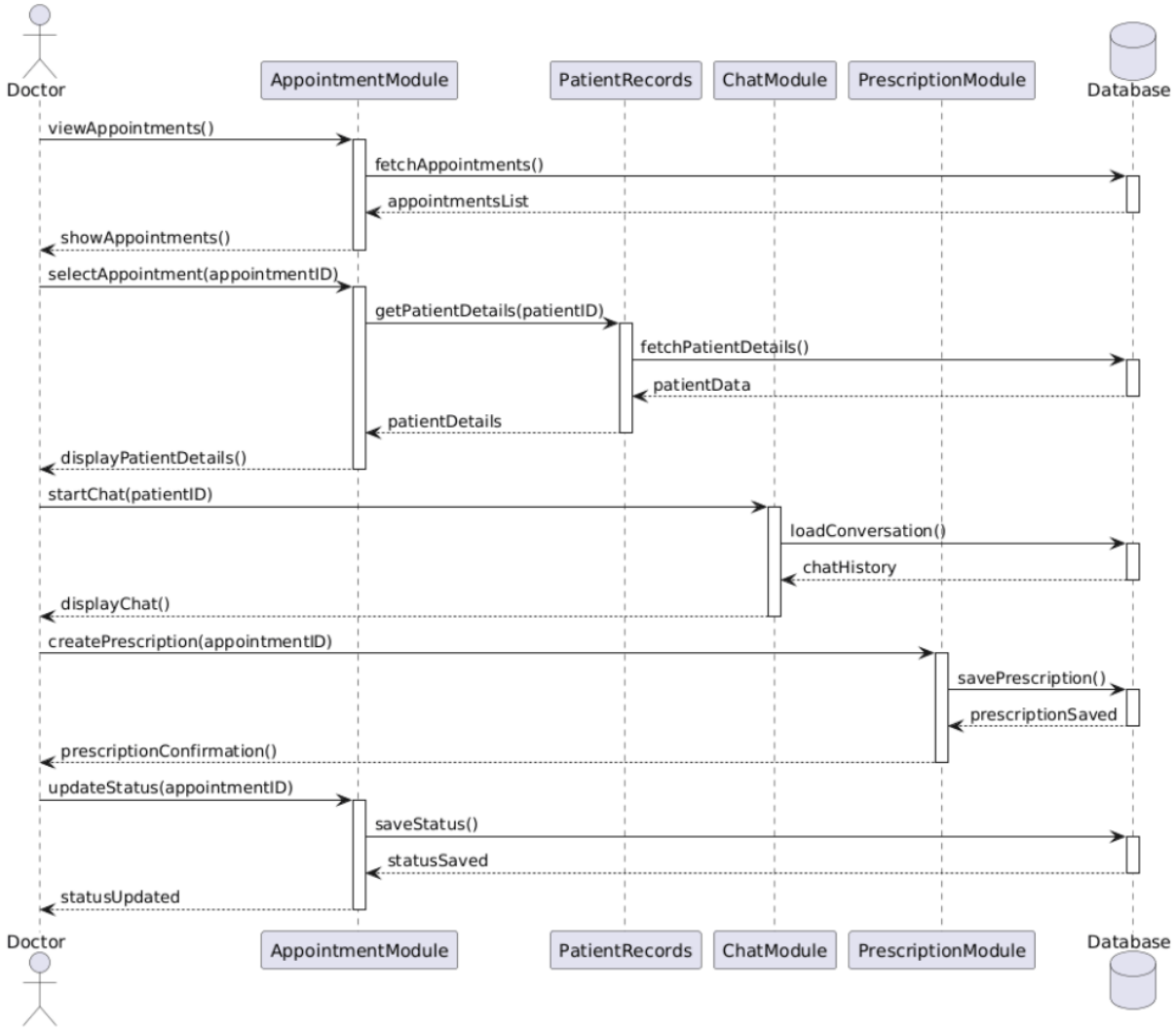

Figure 4.7 Doctor Sequence Diagram

Figure 4.8 presents the Admin Sequence Diagram, which explains how the administrator interacts with different system modules to manage operational and regulatory tasks. The admin reviews pending doctor registrations, approves or rejects applications, manages user accounts, moderates reviews, generates system reports through the Reports Module, and updates system configurations through the System Settings Module. Each operation involves interaction with a specific management module and the central Firestore database.

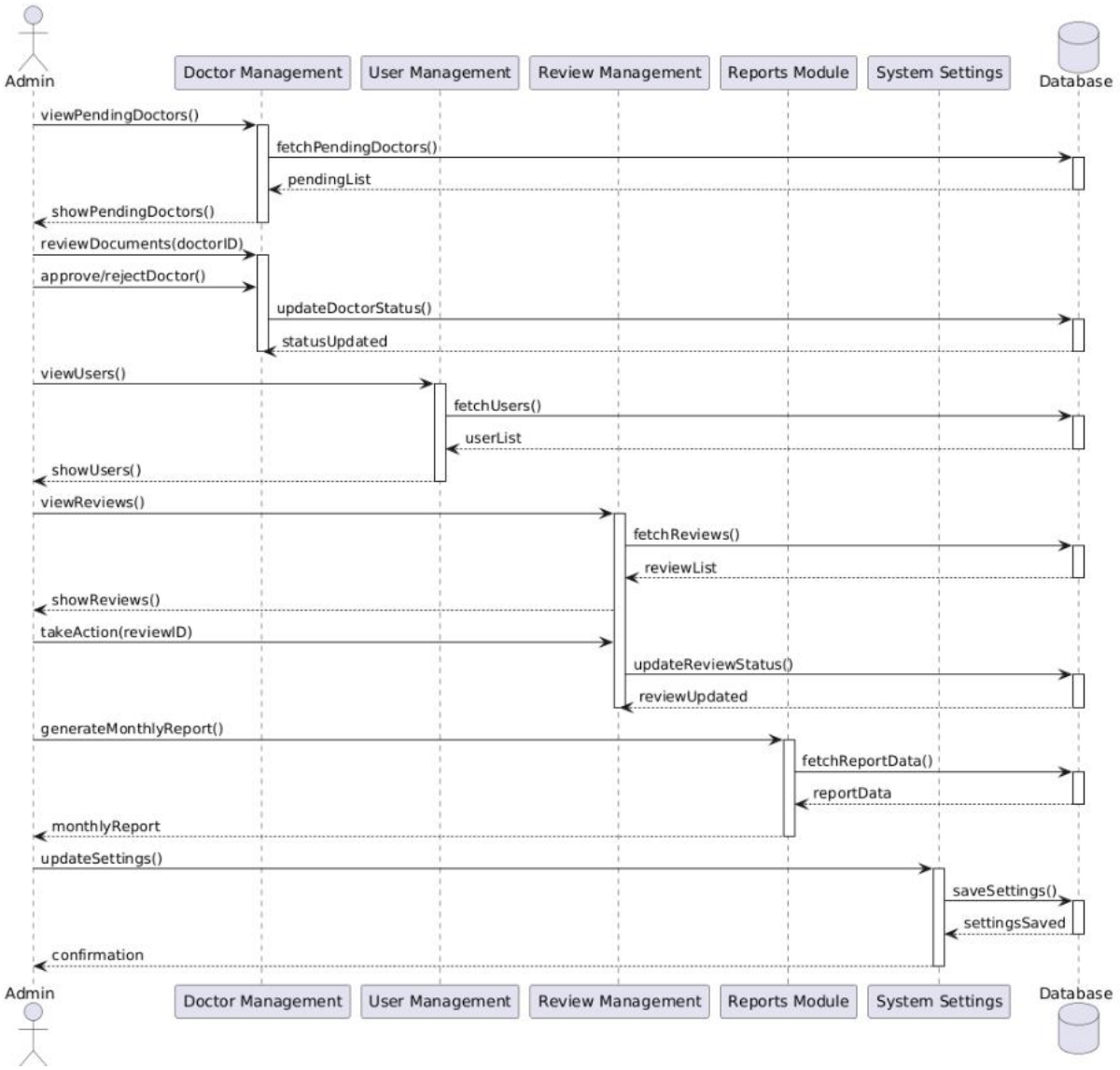

Figure 4.8 Admin Sequence Diagram

5. Implementation

This chapter discusses the implementation details of the AidLink system, including the core algorithms used to perform major operations, the external APIs integrated into the system, and the user interface screens developed across all three platforms.

5.1 Algorithm

This section presents the natural language explanation and pseudocode of the core algorithms used to implement the major functional modules of AidLink.

5.1.1 Doctor Search and Filtering Algorithm

In AidLink, doctor discovery is implemented through a live Firestore stream rather than a one-time query, so results update in real time as doctor records change. The patient first selects a specialization category and the system streams all doctor documents whose status field equals "approved", then filters that stream client-side by matching the doctor's specialization field against the selected category after normalizing case and whitespace to avoid mismatches. For proximity-based discovery, the patient's current GPS coordinates are obtained via the Geolocator package, and the distance to each doctor's clinic location is computed using the Geolocator package's built-in distance calculation based on the Haversine formula. Results are then sorted in ascending order by distance.

Pseudocode:

```
FUNCTION streamDoctorsByCategory(category):
   doctorStream = Firestore.collection("doctors")
        .where("status", isEqualTo: "approved")
        .snapshots()
ON each snapshot:
filtered = []
FOR each doctor IN snapshot:
        IF normalize(doctor.specialization) == normalize(category):
        filtered.add(doctor)
UPDATE UI with filtered
RETURN doctorStream

FUNCTION findNearbyProviders(patientLocation, clinicList):
results = []
FOR each clinic IN clinicList:
distanceMeters = Geolocator.distanceBetween(
        patientLocation.lat, patientLocation.lng,
        clinic.lat, clinic.lng
)
      clinic.distanceKm = distanceMeters / 1000
results.add(clinic)
```

SORT results ASCENDING by distanceKm
RETURN results

5.1.2 Real-Time Appointment Booking Algorithm

Booking conflicts in AidLink are prevented at write time rather than through a separate slot status field. When a patient submits a booking request, the system queries existing appointments for the selected doctor on the requested date and time before creating the new appointment document. If a conflicting appointment is found, the booking is rejected. Otherwise, the appointment is written to Firestore with an initial status of "pending" and the doctor receives an in-app notification document stored in Firestore.

Pseudocode:

```
FUNCTION bookAppointment(patientId, doctorId, date, time, visitType):
existing = Firestore.collection("appointments")
        .where("doctorId", isEqualTo: doctorId)
        .where("date", isEqualTo: date)
        .where("time", isEqualTo: time)
        .where("status", in: ["pending", "approved"])
        .get()

IF existing is NOT empty:
      RETURN error("This time slot is no longer available")

appointment = {
patientId, doctorId, date, time, visitType,
status: "pending",
createdAt: serverTimestamp()
}
newDoc = Firestore.collection("appointments").add(appointment)

NotificationService.createNotification(
recipientId: doctorId,
type: "appointment_requested",
title: "New appointment request",
data: { appointmentId: newDoc.id }
)

RETURN newDoc
```

5.1.3 Doctor Verification Algorithm

When a new doctor registers, their account is created with a status of "pending" and uploaded credential files including CNIC, medical license, and degree certificate are stored either as hosted URLs via imgbb or as inline Base64 data URIs directly in Firestore as a fallback. The administrator reviews these documents from the admin panel and approves or rejects the application, which updates the status field and triggers a Firestore notification to the doctor along with an activity log entry.

Pseudocode:

```
FUNCTION verifyDoctor(doctorId, decision, adminId):
doctorDoc = Firestore.collection("doctors").doc(doctorId).get()

IF decision == "approve":
Firestore.update(doctorDoc, {
        status: "approved",
        approvedAt: serverTimestamp(),
        approvedBy: adminId
})
notificationType = "doctor_approved"
ELSE:
Firestore.update(doctorDoc, {
        status: "rejected",
        rejectionReason: reasonText,
        reviewedBy: adminId
})
notificationType = "doctor_rejected"

NotificationService.createNotification(
recipientId: doctorId,
type: notificationType,
title: "Verification update",
body: "Your verification request has been " + decision
)

AdminActivityService.log(
action: "doctor_" + decision,
targetId: doctorId,
actorId: adminId
)
```

5.1.4 Prescription Creation and Retrieval Algorithm

AidLink stores prescriptions as structured Firestore documents at creation time. The medicine list, dosage, frequency, diagnosis, and advice fields are saved as plain document fields rather than as a pre-generated file. The PDF is generated on demand on the client side only when the patient or doctor chooses to view, download, or share the prescription, using the Flutter pdf package. This approach keeps prescription creation instantaneous and avoids any dependency on cloud file storage.

Pseudocode:

```
FUNCTION createPrescription(appointmentId, doctorId, patientId,
                medicines, diagnosis, advice, followUpDate):
prescription = {
appointmentId, doctorId, patientId,
medicines, diagnosis, advice, followUpDate,
createdAt: serverTimestamp()
}
newDoc = Firestore.collection("prescriptions").add(prescription)

NotificationService.createNotification(
recipientId: patientId,
type: "prescription_added",
title: "New prescription available"
)

RETURN newDoc

FUNCTION exportPrescriptionAsPdf(prescriptionId):
data = Firestore.collection("prescriptions").doc(prescriptionId).get()
pdfBytes = generatePdfDocument(data)
RETURN pdfBytes
```

5.1.5 In-App Chat Message Delivery and Read-Receipt Algorithm

AidLink's chat module uses Firestore real-time listeners for instant message delivery. Real-time delivery is achieved through two mechanisms: first, an in-app notification document is created in Firestore for the recipient on every message sent, which the recipient's notification stream picks up; second, read-receipt state is tracked per message and rendered as delivery tick indicators, updated via targeted Firestore field writes rather than rewriting the entire document.

Pseudocode:

```
FUNCTION sendMessage(roomId, senderId, text):
message = {
```

```
senderId, text,
createdAt: serverTimestamp(),
delivered: false,
readBy: []
}
Firestore.collection("rooms").doc(roomId)
        .collection("messages").add(message)

room = Firestore.collection("rooms").doc(roomId).get()
FOR each participant IN room.participants WHERE participant != senderId:
Firestore.update(room, { "unreadCounts." + participant: increment(1) })
NotificationService.createNotification(
        recipientId: participant,
        type: "chat_message",
        title: "New message",
        body: truncate(text, 120)
)

FUNCTION onChatScreenOpened(roomId, currentUserId, otherUserId):
Firestore.update(room, { "unreadCounts." + currentUserId: 0 })
unreadMessages = Firestore.collection("rooms").doc(roomId)
                .collection("messages")
                .where("senderId", isEqualTo: otherUserId)
                .get()
FOR each message IN unreadMessages:
IF currentUserId NOT IN message.readBy:
        Firestore.update(message, {
        readBy: arrayUnion([currentUserId]),
        delivered: true
        })

FUNCTION renderTickIcon(message, currentUserId, otherUserId):
IF currentUserId IN message.readBy:
RETURN blueDoubleCheck
ELSE IF message.delivered == true:
RETURN greyDoubleCheck
ELSE:
RETURN greySingleCheck
```

5.2 External APIs

AidLink integrates several external APIs and backend services to deliver its core functionality.

Table 5.1 presents the complete details of all APIs used in the project, including their purpose and the specific functions or classes in which they are utilized.

Table 5.1 Details of APIs Used in the Project

Name of API	Description of API	Purpose of Usage	Function/Class Used In
Firebase Authentication API	Provides secure user authentication and identity management	Used to register, login, and authenticate Patients, Doctors, and Admins	User.login(), User.register()
Firebase Firestore API	NoSQL real-time cloud database	Used to store and retrieve all application data including users, appointments, prescriptions, and chat messages in real time	Appointment.confirm(), Doctor.updateSchedule()
Firebase Cloud Storage API	Cloud-based file storage service	Used to store doctor credential documents and prescription PDF files	Prescription.generatePDF(), Doctor.uploadCredentials()
Firebase Cloud Functions API	Serverless backend execution environment	Used to execute backend logic such as sending notifications upon appointment confirmation	sendNotification(), onAppointmentCreated()
Firebase Cloud Messaging (FCM) API	Push notification delivery service	Used to deliver real-time push notifications for appointments, chat messages, and system alerts	Notification.send(), ChatMessage.send()
Google Maps API	Location and mapping service	Used to display doctor locations on an interactive map and calculate distance for location-based doctor search	searchDoctors(), Doctor.clinicLocation
Google Geolocation API	Device location retrieval service	Used to retrieve the patient's current location for proximity-based doctor search	searchDoctors(), getPatientLocation()

5.3 User Interface

This section presents the user interface screens implemented across the three platforms of AidLink: the Patient Mobile Application, the Doctor Web Portal, and the Admin Web Portal. Each screen is described individually below, illustrating the core functionality available to each user role.

5.3.1 Patient Mobile Application

Figure 5.1 shows the splash screen of the AidLink mobile application, displayed briefly when the application is launched.

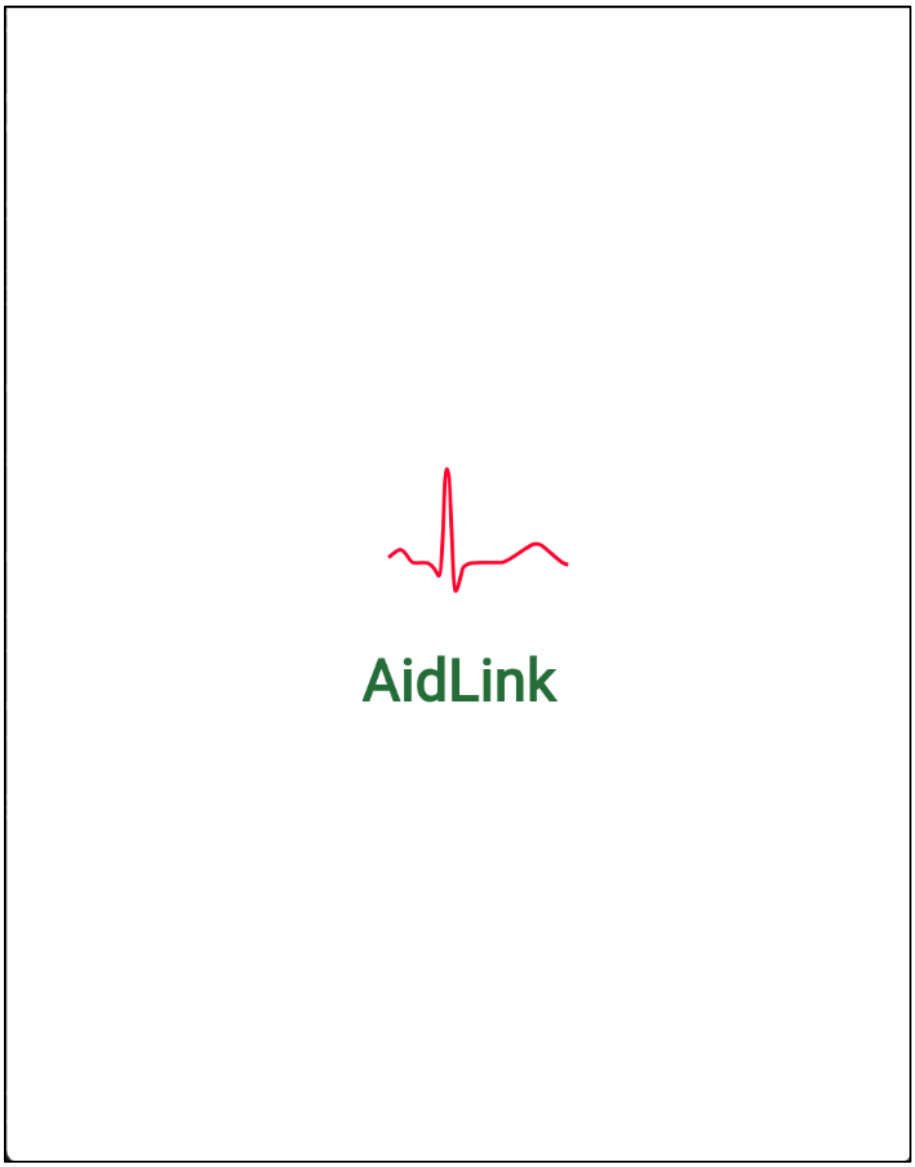

Figure 5.1: Splash screen

Figure 5.2 shows the login screen, which allows registered patients and doctors to authenticate using their email and password.

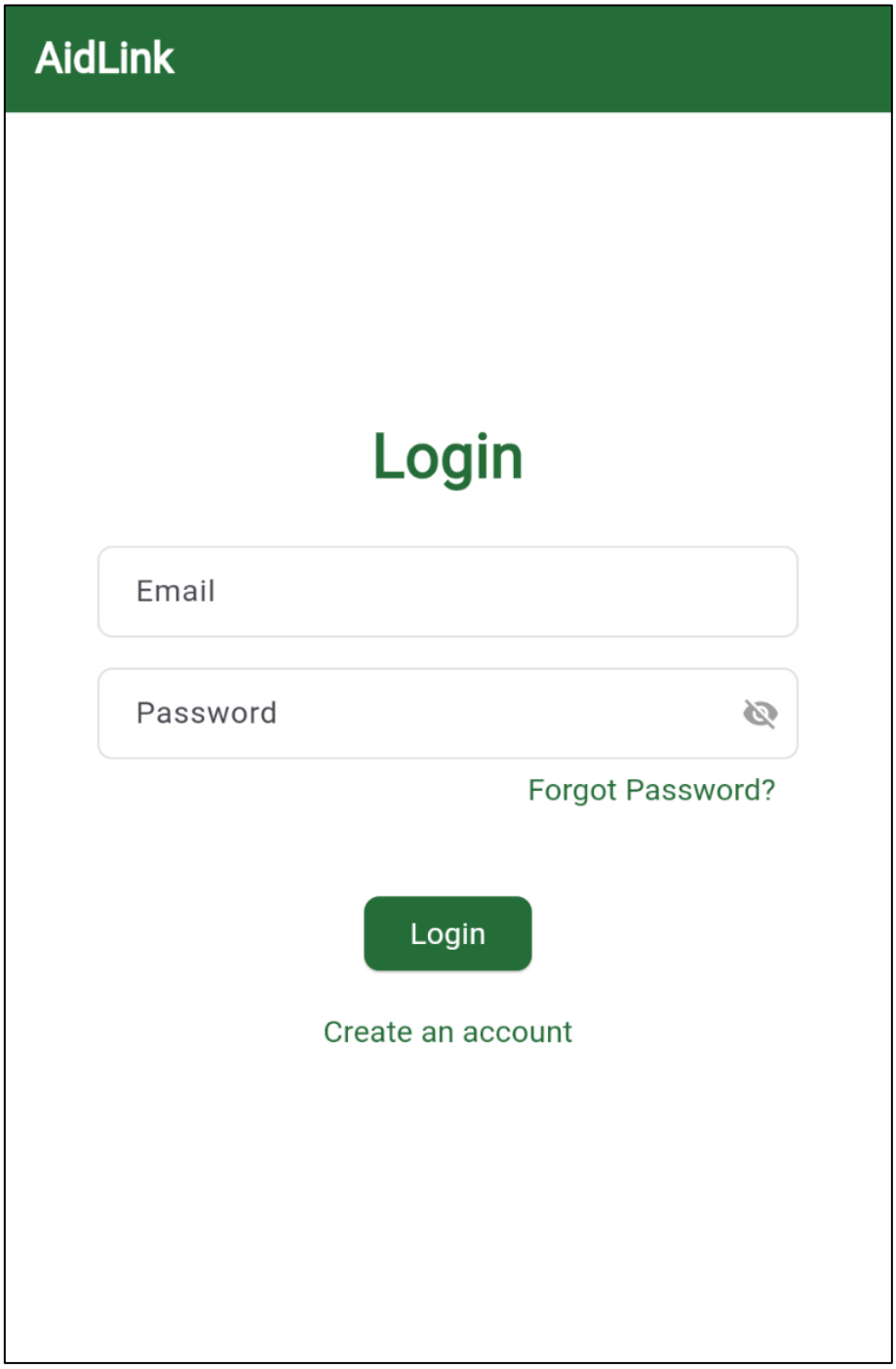

Figure 5.2: Login screen

Figure 5.3 shows the registration screen, where a new user selects their role as either Patient or Doctor and enters their personal details to create an account.

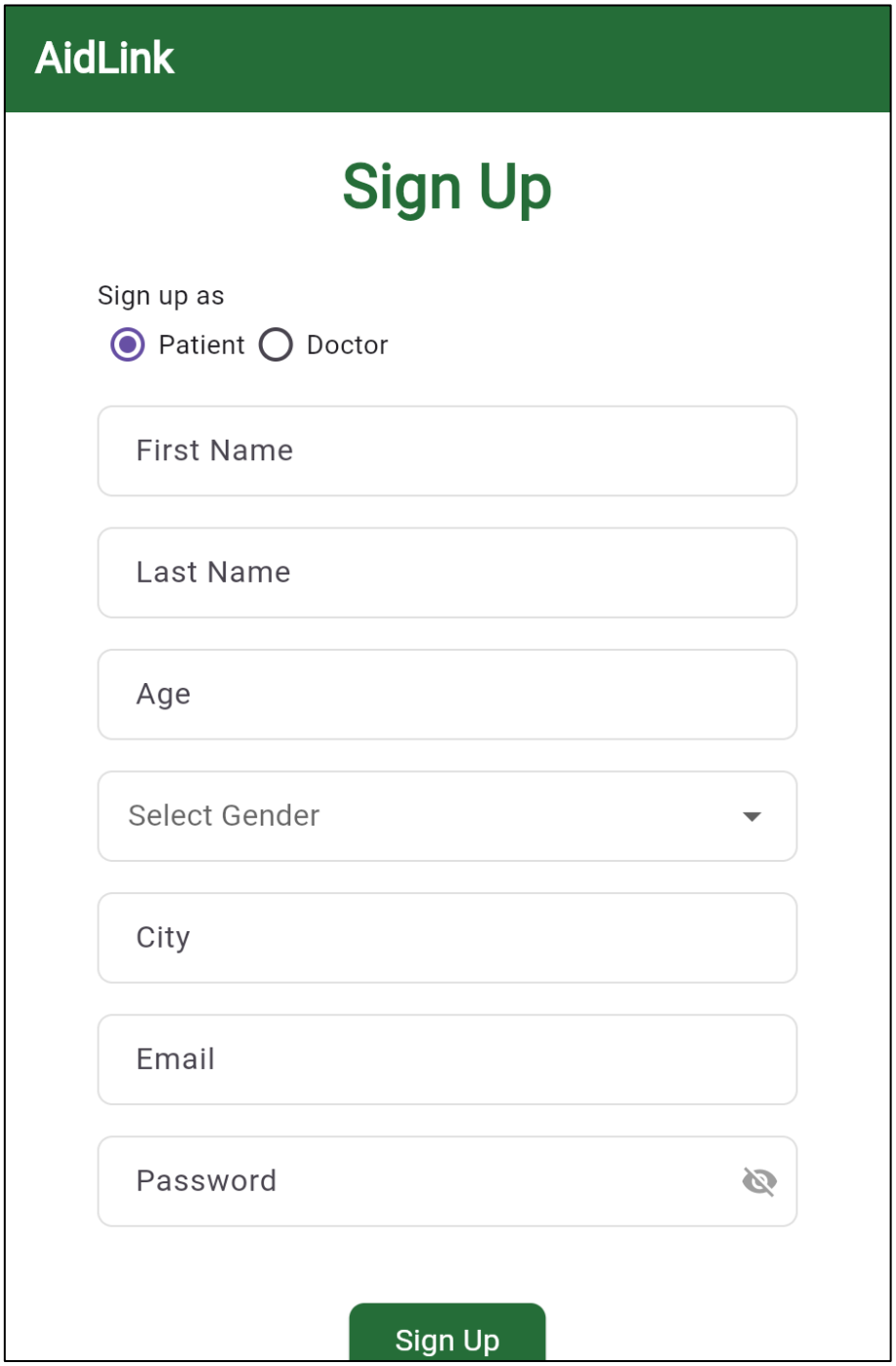

Figure 5.3: Registration screen

Figure 5.4 shows the patient home screen, which displays quick access cards for managing appointments, browsing doctors, and tracking prescriptions, along with a search bar and doctor specialization categories.

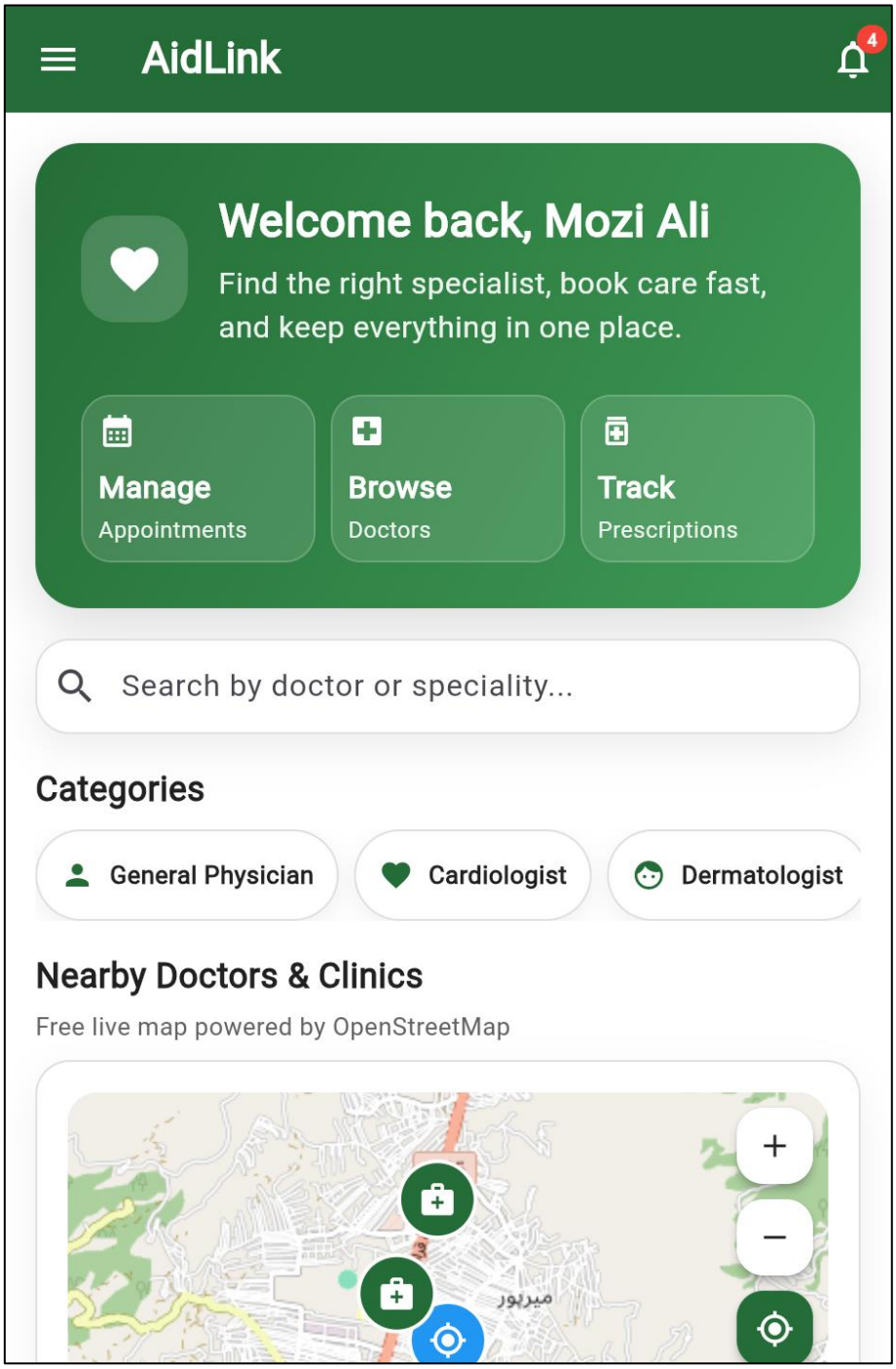

Figure 5.4: Patient home screen

Figure 5.5 shows the doctor search results screen, where matching doctors are displayed based on the patient's search query.

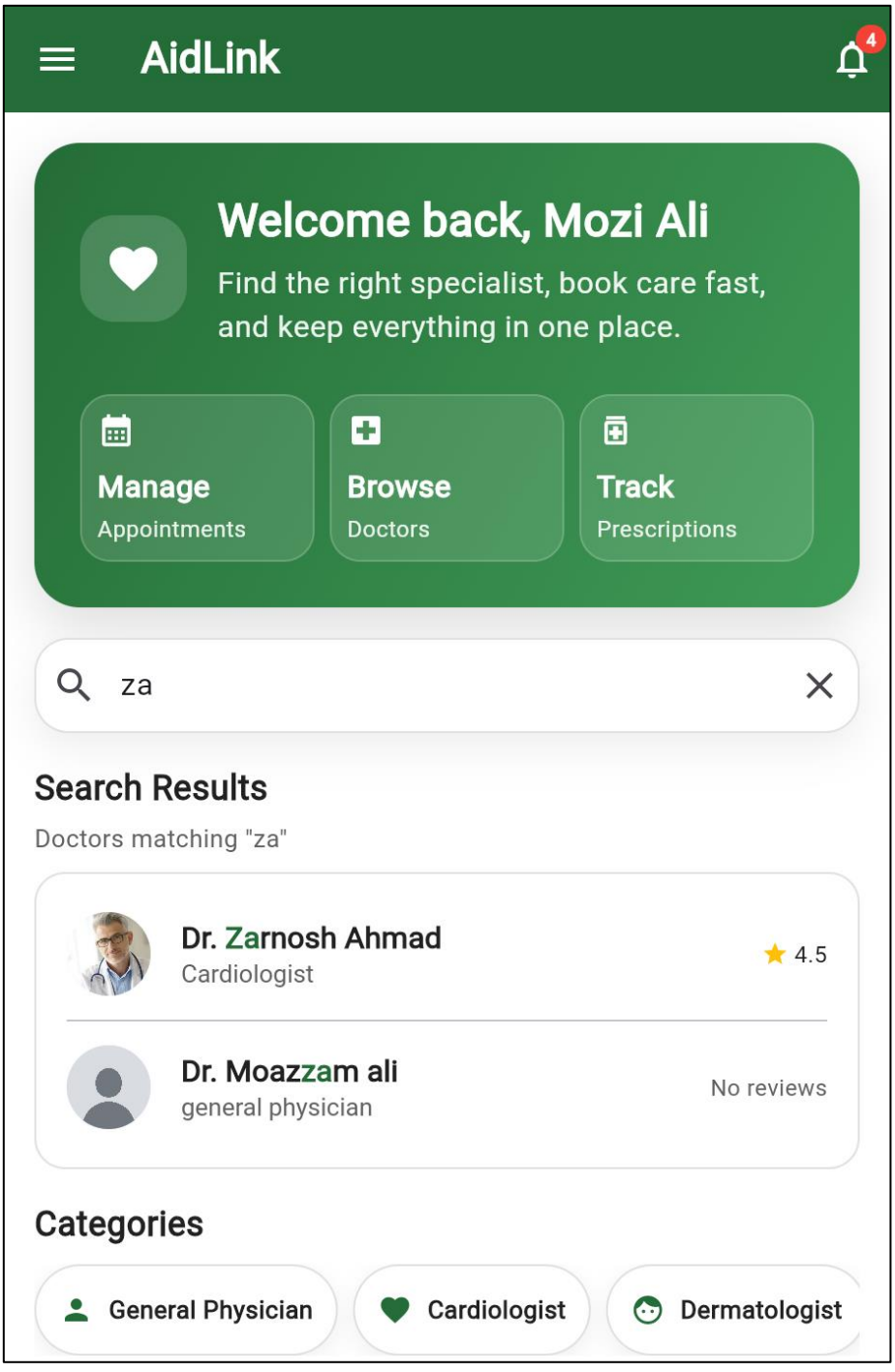

Figure 5.5: Doctor search results screen

Figure 5.6 shows the doctor profile screen, displaying the selected doctor's specialization, experience, consultation availability, and patient reviews.

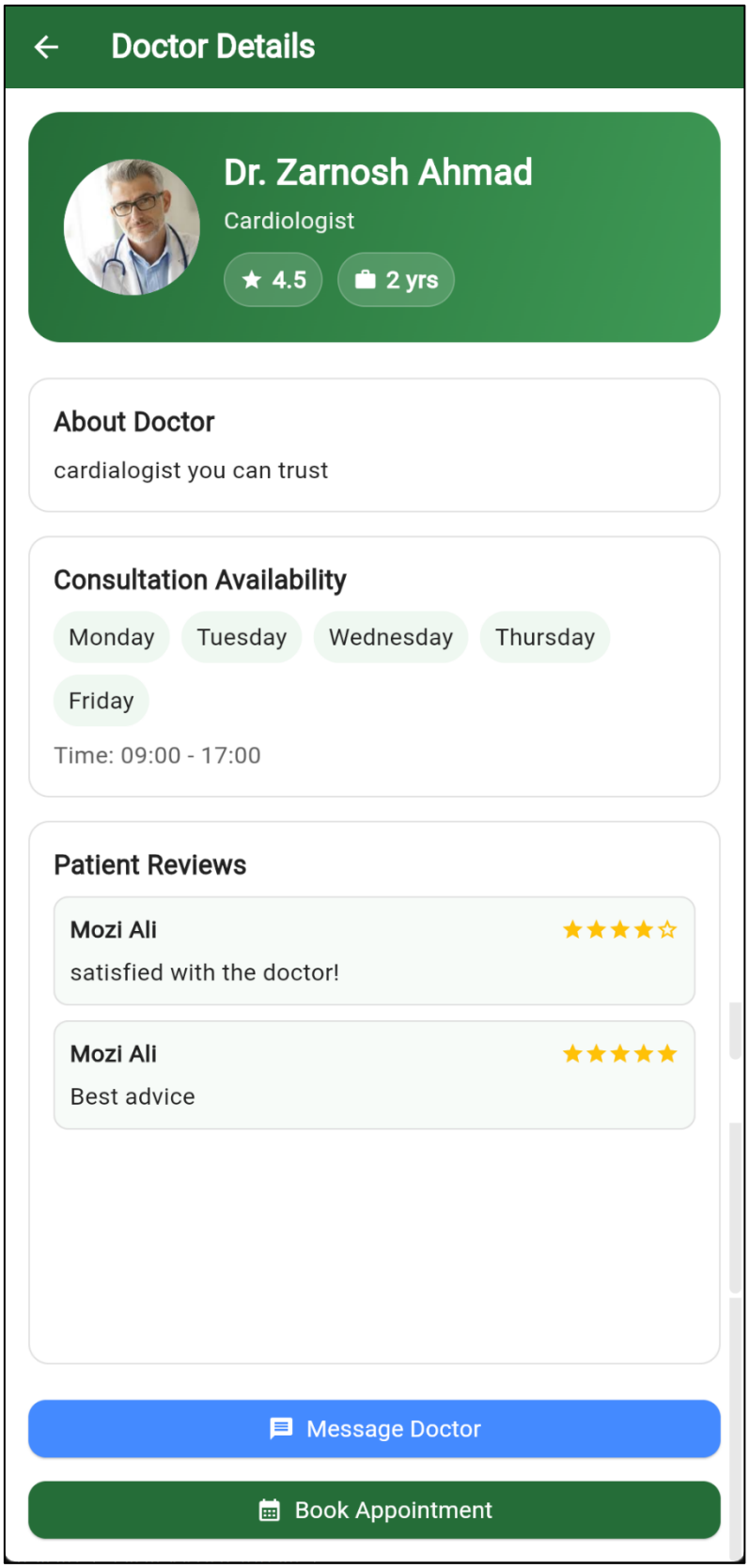

Figure 5.6: Doctor profile screen

Figure 5.7 shows the appointment booking screen, where the patient selects a doctor, chooses an available date and time slot, and submits relevant symptoms before confirming the appointment.

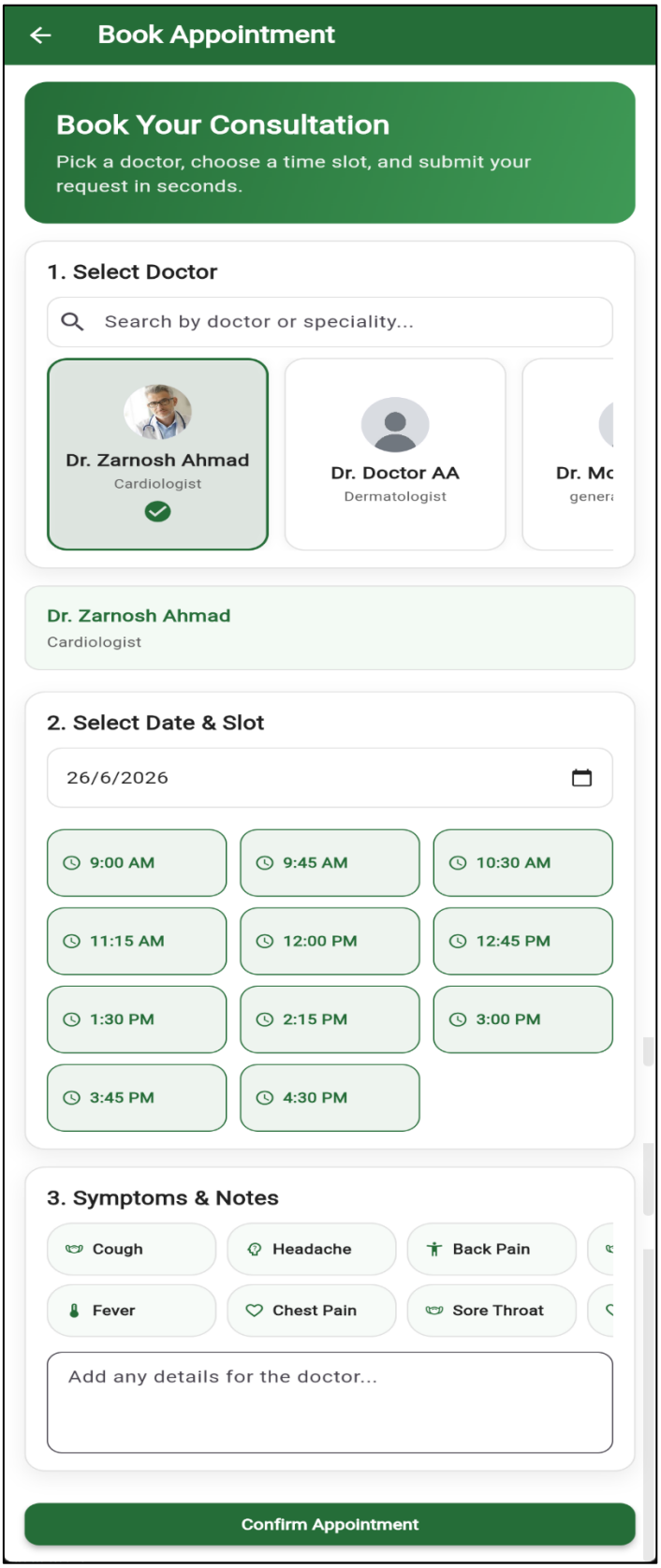

Figure 5.7: Appointment booking screen

Figure 5.8 shows the map-based doctor listing screen, which displays nearby doctors and clinics using the Google Maps API along with the patient's current location.

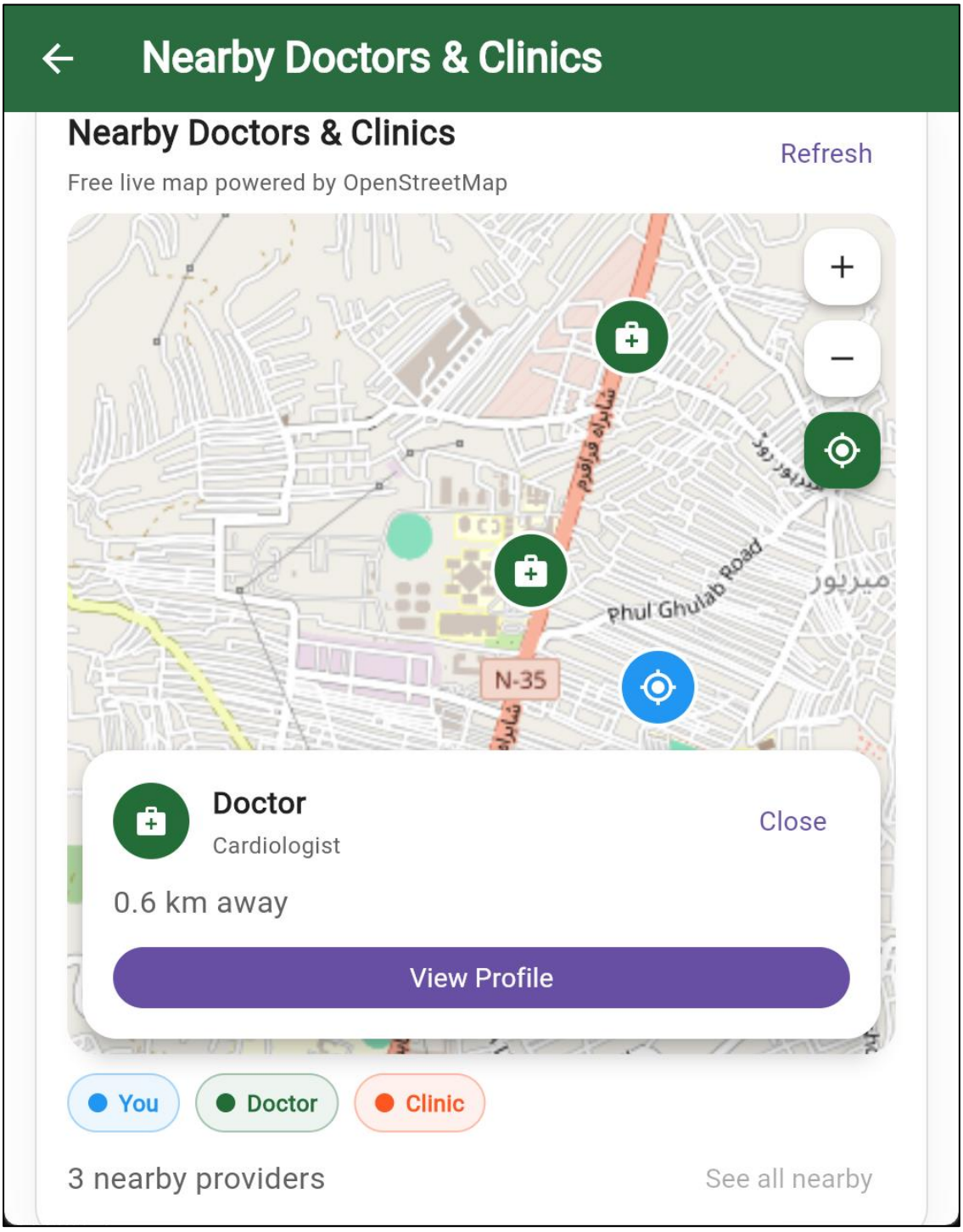

Figure 5.8: Map-based doctor listing screen

Figure 5.9 shows the in-app chat screen, allowing the patient to communicate directly with the doctor for follow-ups and consultation-related queries.

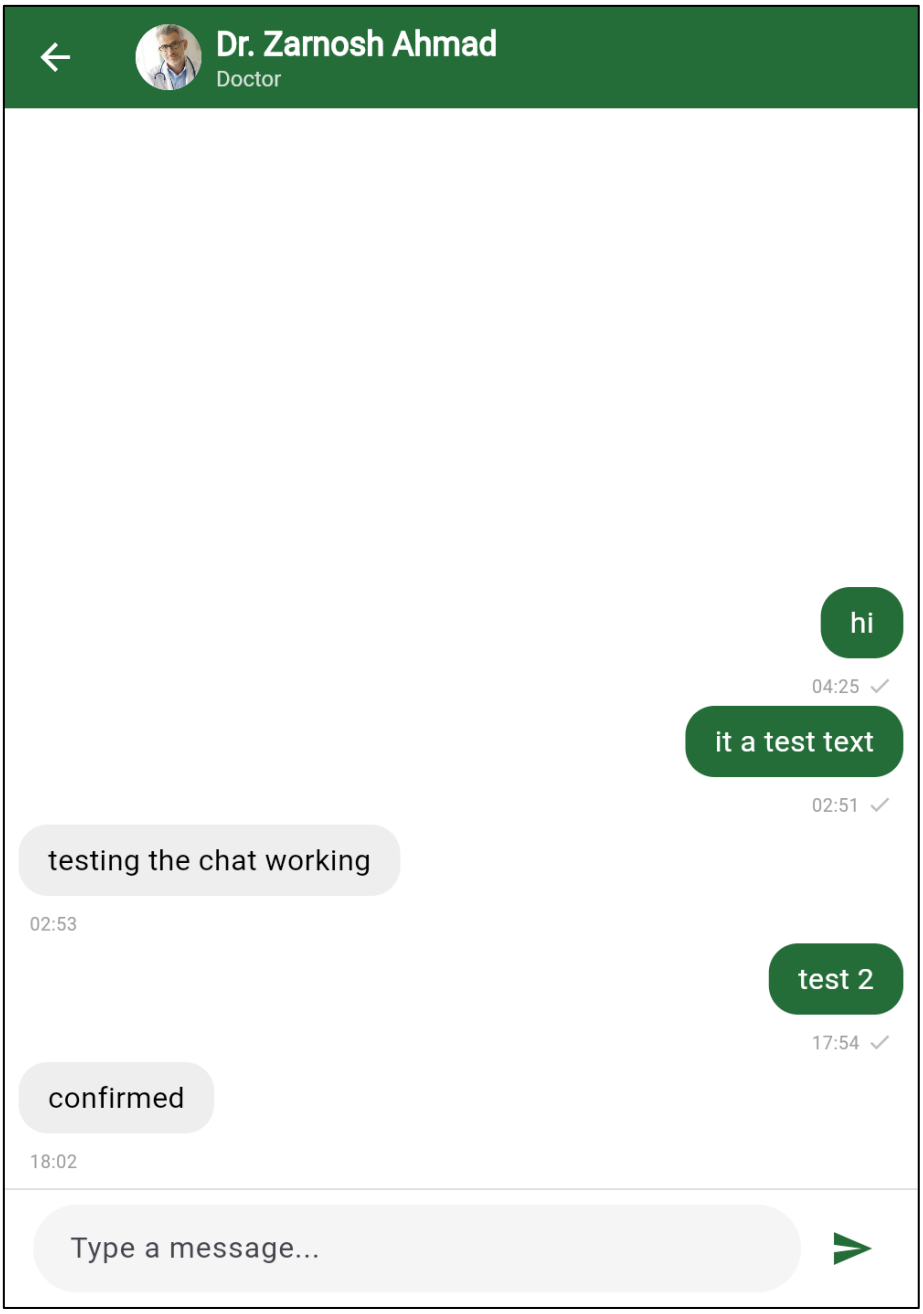

Figure 5.9: In-app chat screen

Figure 5.10 shows the prescriptions screen, where the patient can view and download all prescriptions issued by their doctors.

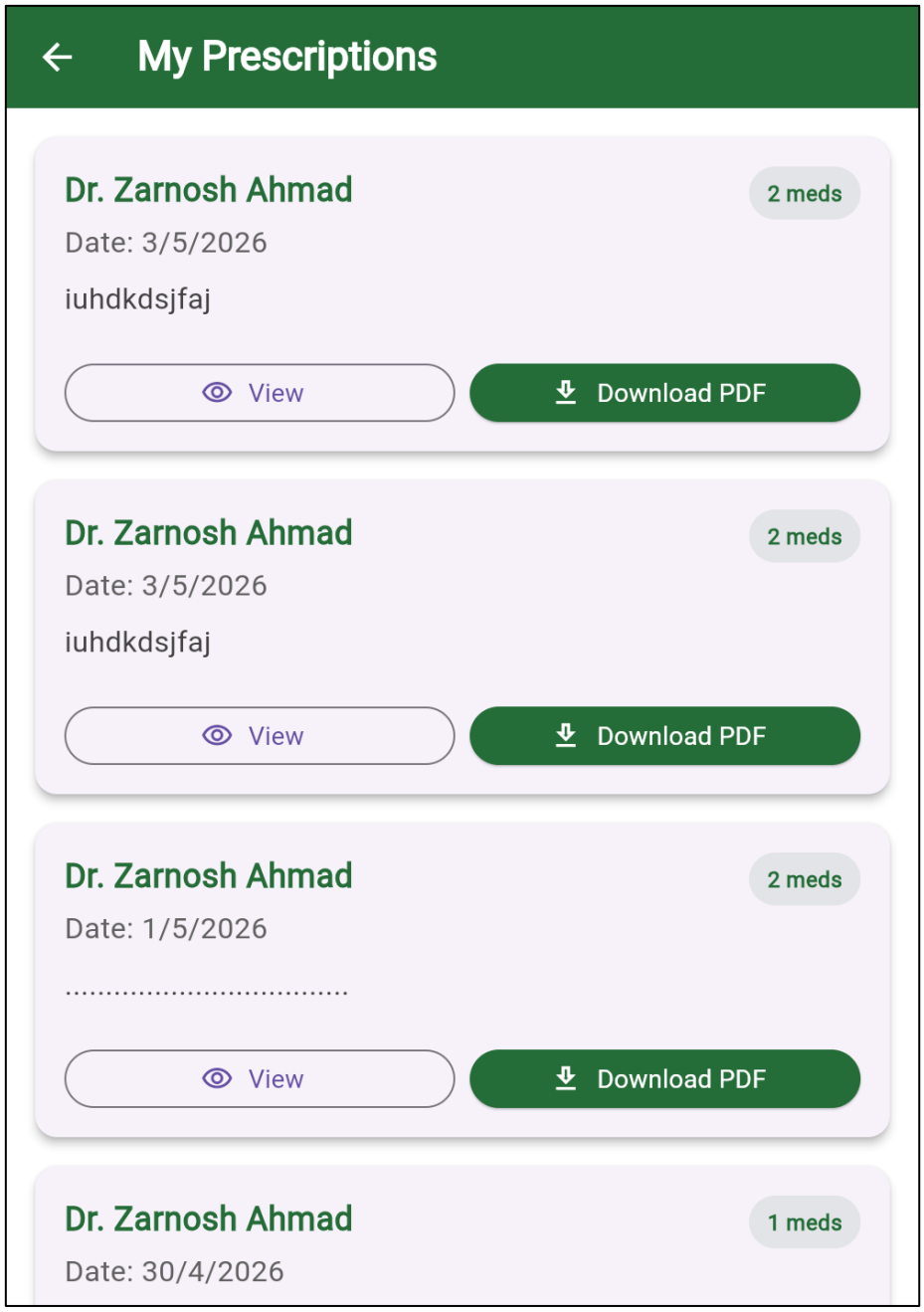

Figure 5.10: Prescriptions screen

Figure 5.11 shows the appointment history screen, displaying the patient's past and cancelled appointments along with their corresponding review status.

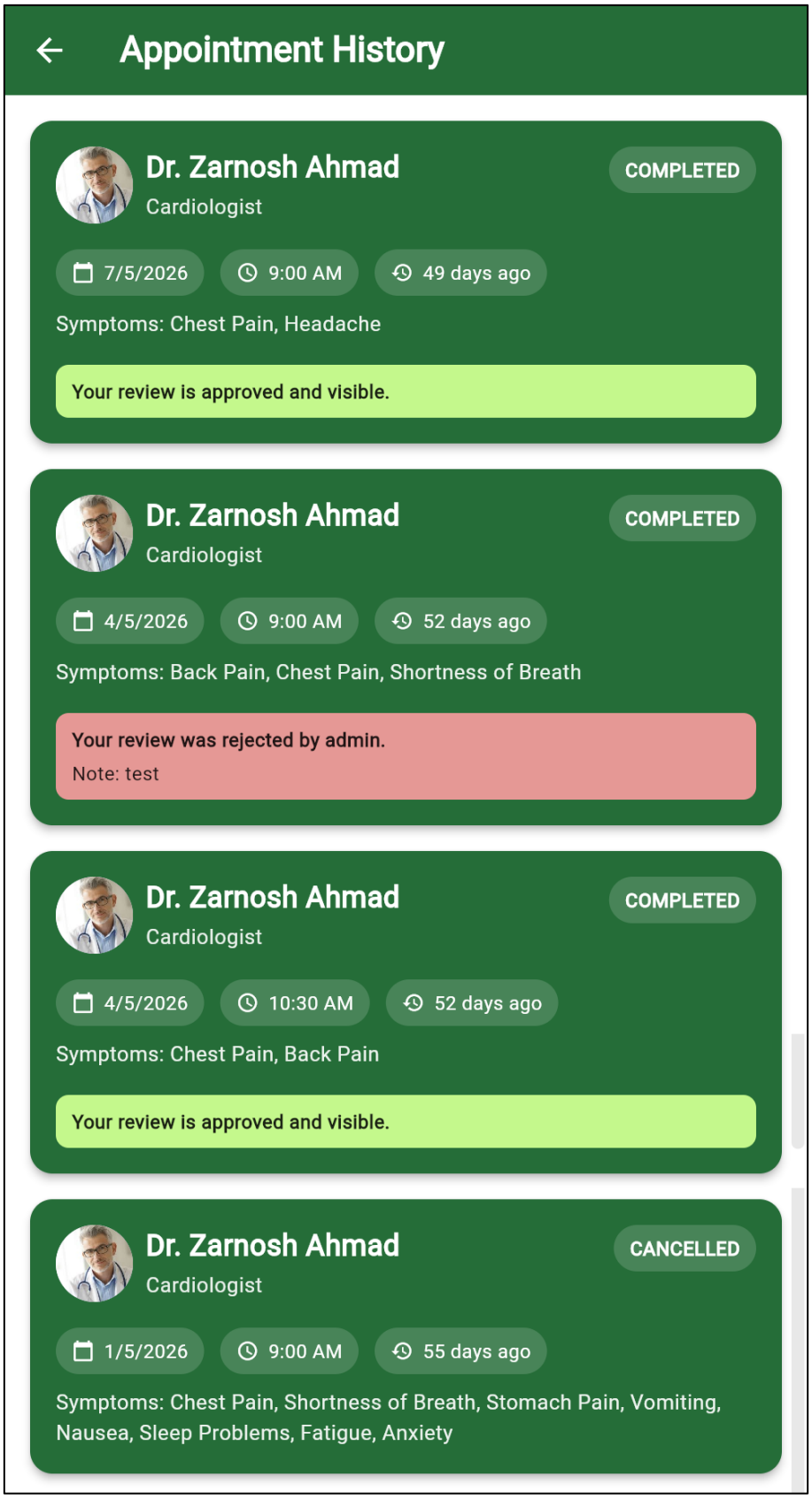

Figure 5.11: Appointment history screen

Figure 5.12 shows the rating and review screen, which allows the patient to rate their consultation experience and submit written feedback for the doctor.

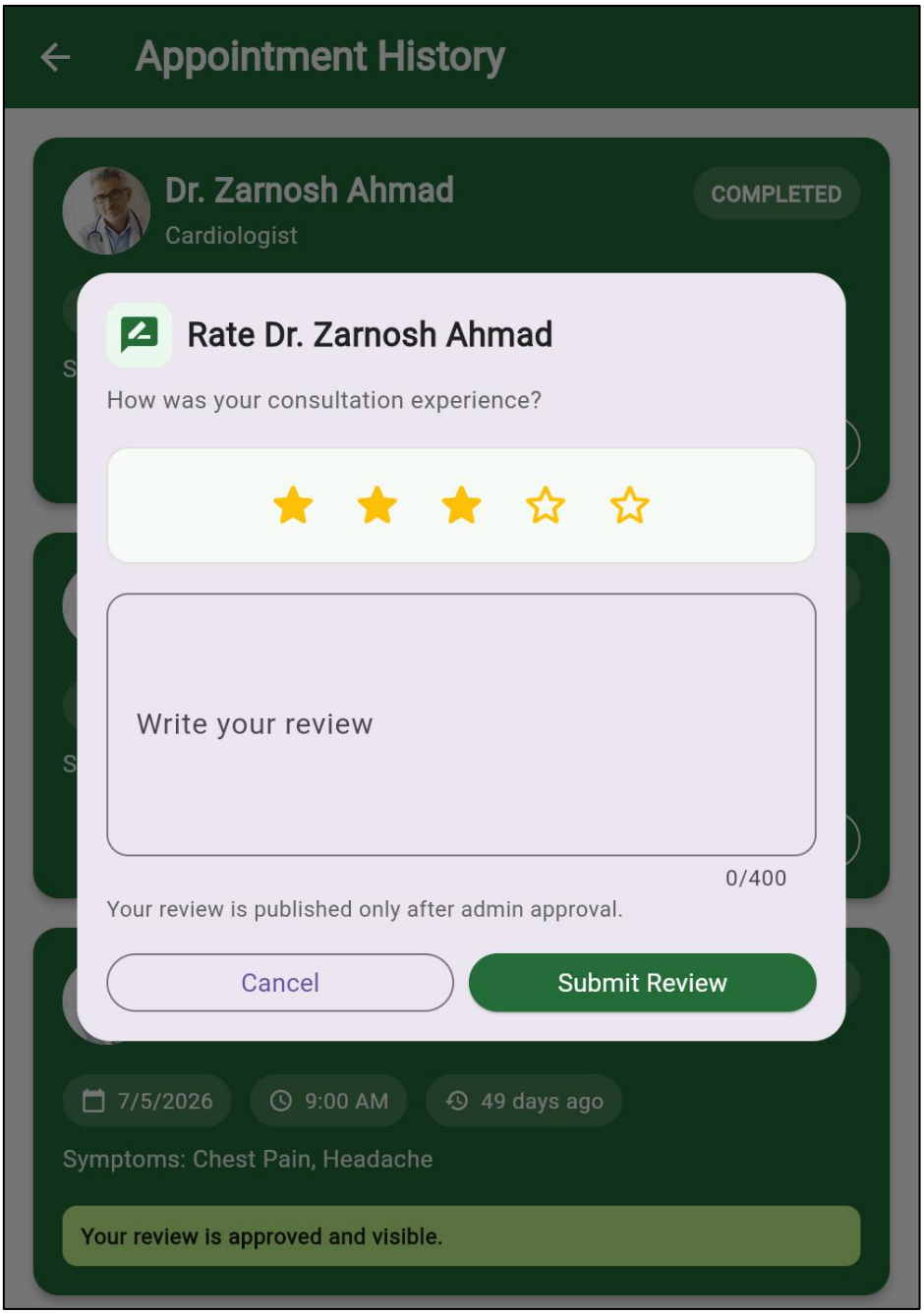

Figure 5.12: Rating and review screen

5.3.2 Doctor Web Portal

Figure 5.13 shows the doctor dashboard, providing an overview of upcoming appointments, pending requests, and recent activity.

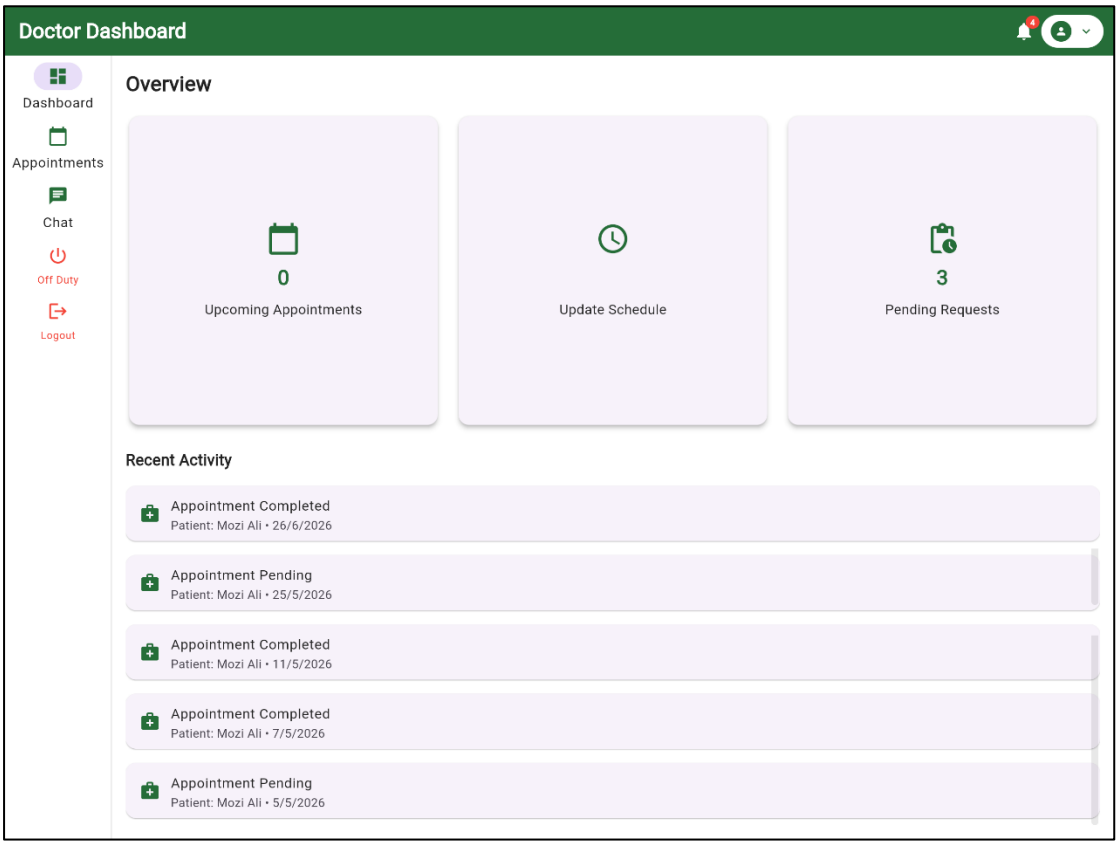

Figure 5.13: Doctor dashboard

Figure 5.14 shows the schedule management screen, where the doctor defines working days, consultation hours, and appointment slot duration.

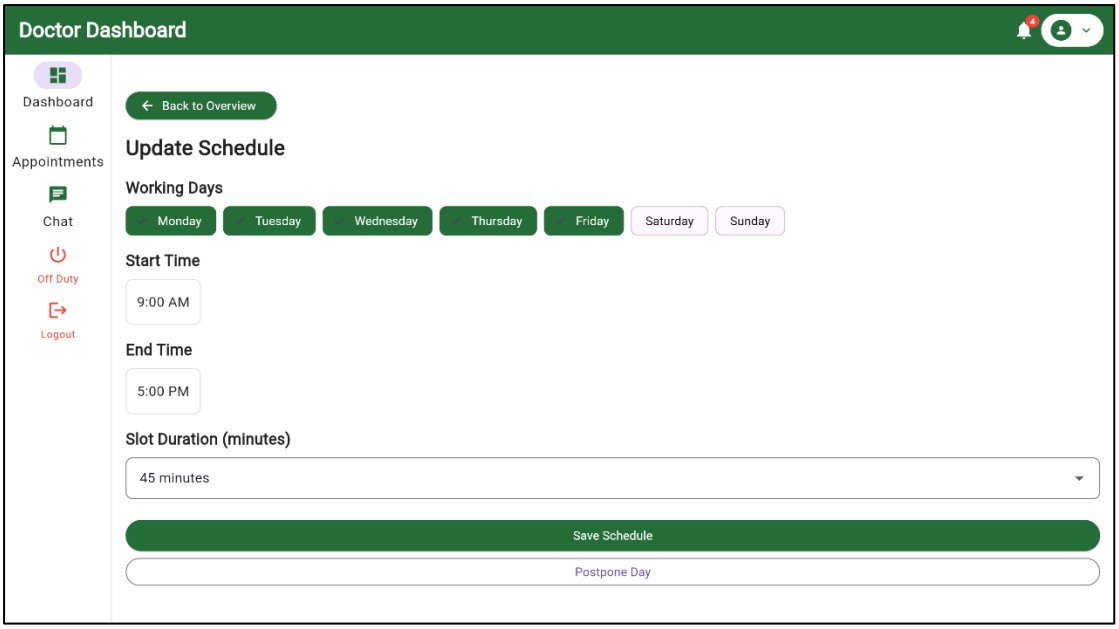

Figure 5.14: Schedule management screen

Figure 5.15 shows the appointments list screen, displaying all patient appointments along with their status and available actions.

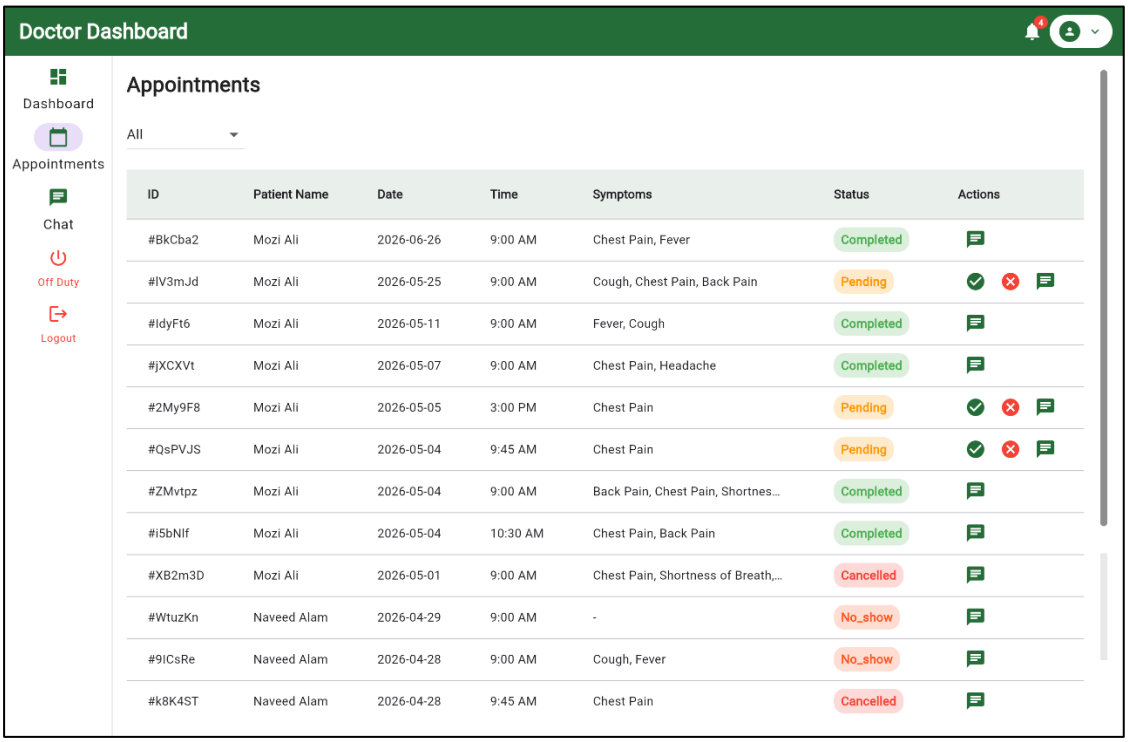

Figure 5.15: Appointments list screen

Figure 5.16 shows the prescription creation screen, where the doctor enters a diagnosis, selects medicines, and saves the completed prescription for the patient.

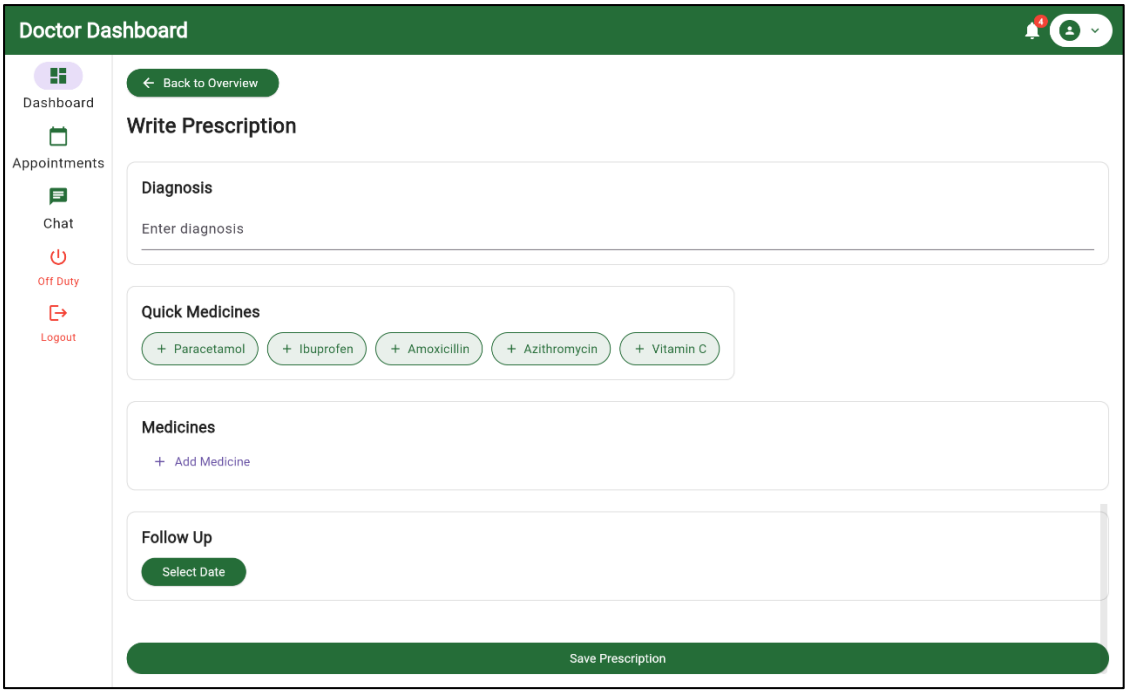

Figure 5.16: Prescription creation screen

5.3.3 Admin Web Portal

Figure 5.17 shows the admin dashboard, displaying key system metrics including pending approvals, completed appointments, and monthly analytics.

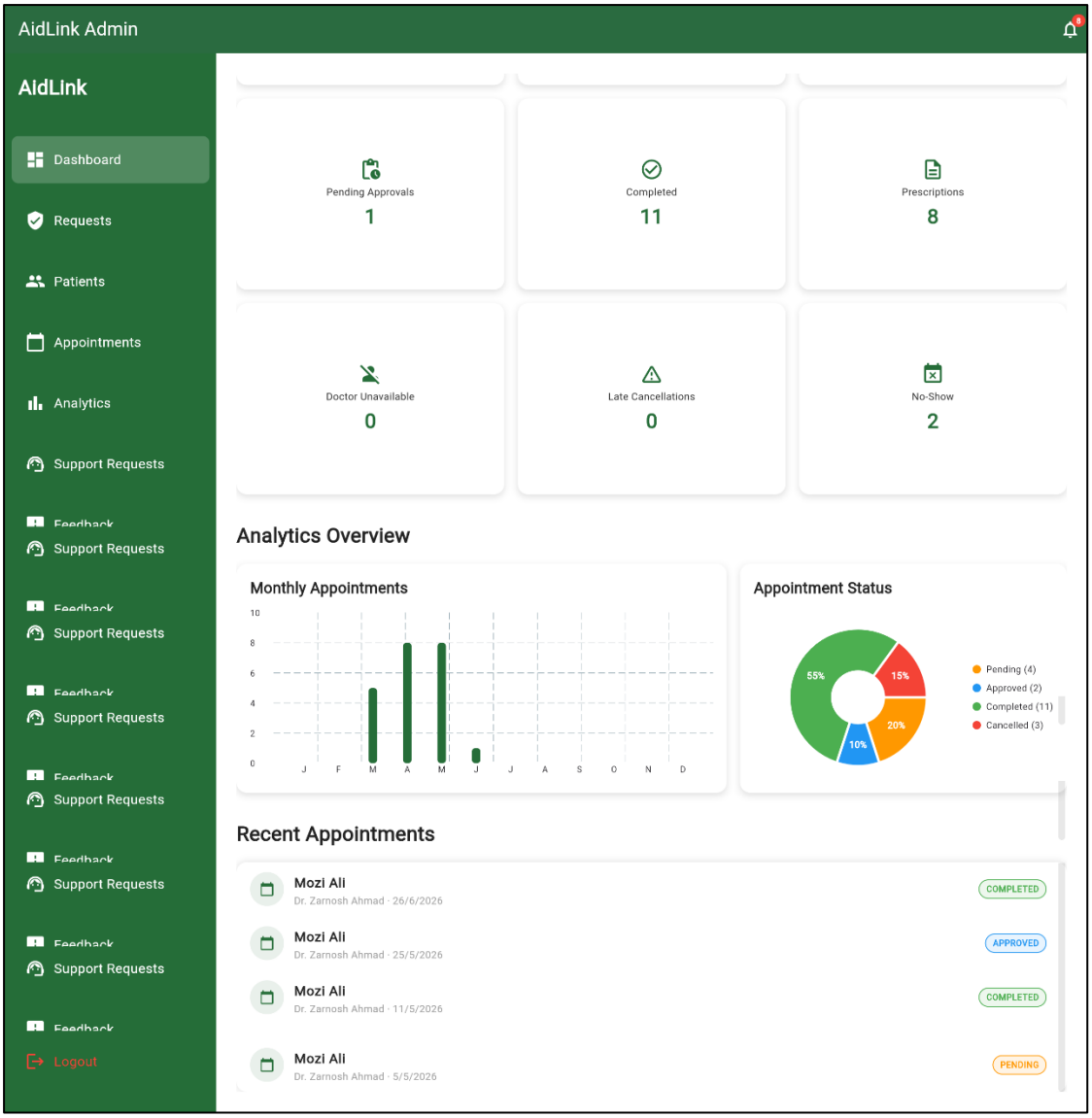

Figure 5.17: Admin dashboard

Figure 5.18 shows the doctor requests screen, where the admin reviews pending, approved, and rejected doctor registration applications.

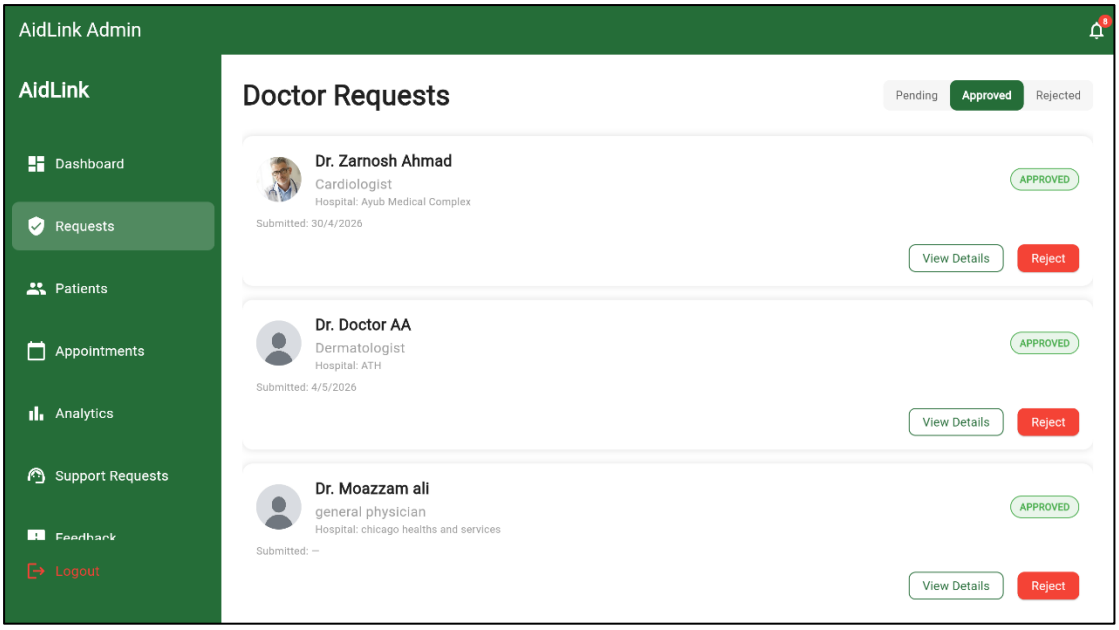

Figure 5.18: Doctor requests screen

Figure 5.19 shows the doctor credential verification screen, displaying the doctor's personal information, professional details, and uploaded documents for admin review.

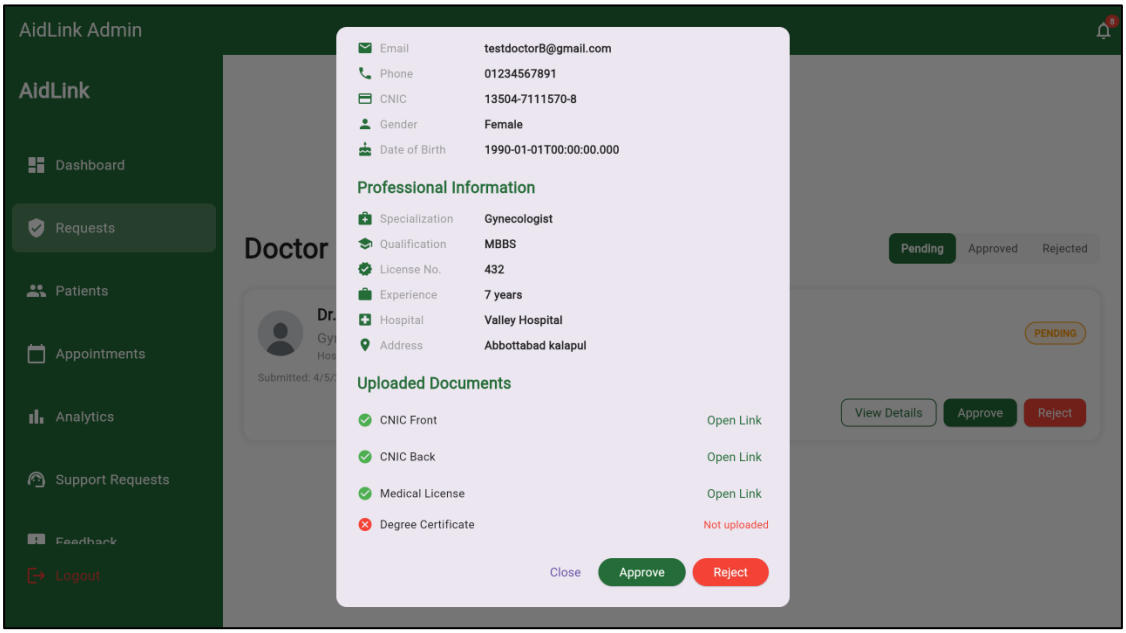

Figure 5.19: Doctor credential verification screen

Figure 5.20 shows the patient management screen, allowing the admin to search, view, and manage all registered patient accounts.

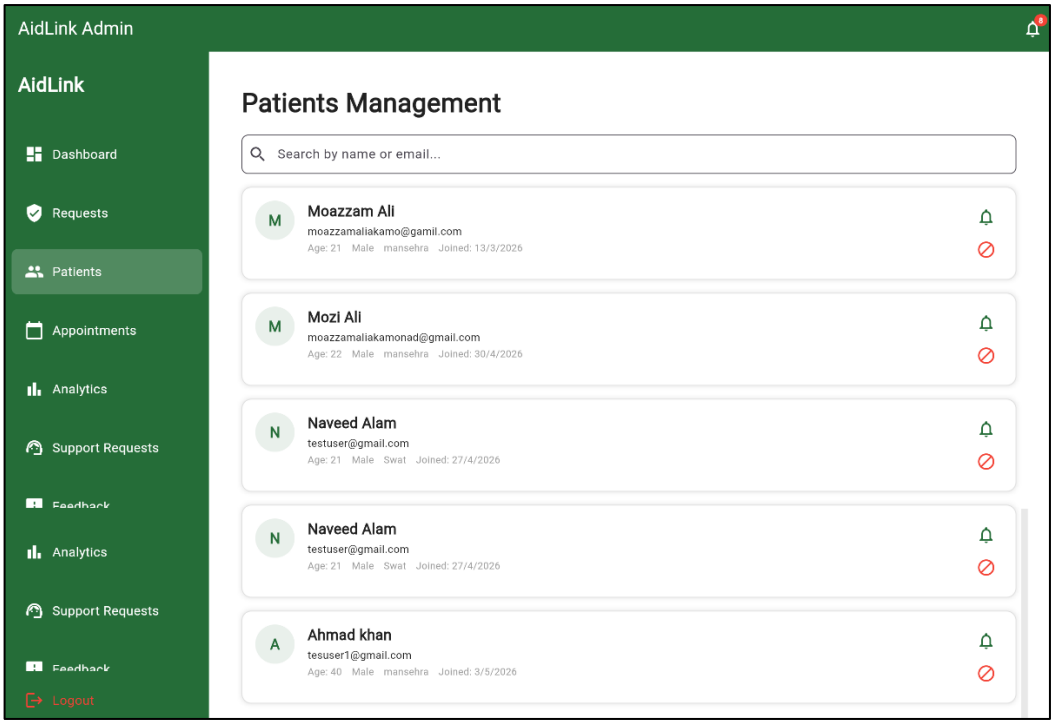

Figure 5.20: Patient management screen

Figure 5.21 shows the analytics screen, presenting system-wide statistics including total appointments, total patients, specialty distribution, and weekly active users.

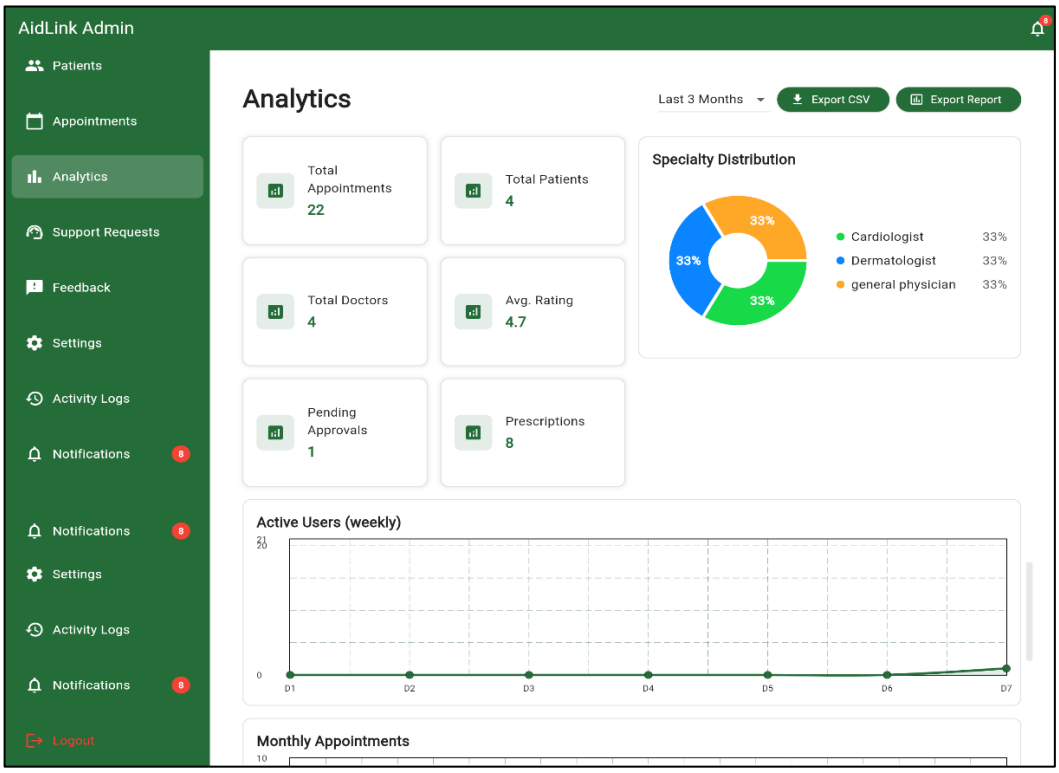

Figure 5.21: Analytics screen

6. Testing and Evaluation

This chapter presents the testing and evaluation process carried out on the AidLink system to ensure that all implemented features function correctly and meet the requirements specified in the Software Requirements Specification. Both manual and automated testing methods were applied, covering unit testing, functional testing, and integration testing.

6.1 Manual Testing

Manual testing was performed across all three interfaces of AidLink to verify that each module behaves as expected under normal and exceptional conditions before deployment.

6.1.1 System Testing

Once the system had been successfully developed, system testing was performed to ensure that AidLink functions as intended and meets the requirements stated in Chapter 3. System testing also helped identify errors that may remain hidden from the end user during isolated module testing. The testing process was divided into unit testing, functional testing, and integration testing, all of which were completed before the system was considered ready for further development and eventual deployment.

6.1.2 Unit Testing

Once individual components of the system had been developed, unit testing was carried out to verify that each isolated function performs correctly.

- **Unit Testing 1:** Patient Login is presented in Table 6.1
- **Testing Objective:** To ensure the login form authenticates patients correctly with valid and invalid credentials.

Table 6.1 Patient Login Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login with correct credentials	Email: patient1@aidlink.com, Password: Patient@123	Logs into Patient Home Screen	Pass
2	Login with incorrect password	Email: patient1@aidlink.com, Password: wrongpass	Displays "Incorrect Password"	Pass
3	Login with unregistered email	Email: notexist@aidlink.com, Password: Patient@123	Displays "Account does not exist"	Pass

- **Unit Testing 2:** Doctor Credential Upload is presented in Table 6.2
- **Testing Objective:** To ensure the credential upload form correctly validates and stores doctor documents.

Table 6.2 Doctor Credential Upload Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Upload valid PDF credential file	File: medical_license.pdf	File uploads successfully and status changes to "Pending Verification"	Pass
2	Upload unsupported file format	File: license.docx	System displays "Unsupported format" message	Pass
3	Upload file exceeding size limit	File: scan.pdf (15MB)	System displays "File too large" message	Pass

- **Unit Testing 3:** Appointment Slot Validation is presented in Table 6.3
- **Testing Objective:** To ensure the system correctly validates real-time slot availability during booking.

Table 6.3 Appointment Slot Validation Unit Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Book an available time slot	Date: 12 July, Time: 10:00 AM	Slot is successfully booked and marked unavailable to other patients	Pass
2	Attempt to book an already booked slot	Date: 12 July, Time: 10:00 AM	System displays "Slot no longer available" message	Pass

6.1.3 Functional Testing

Functional testing was performed after unit testing to evaluate the functionality of each module as a whole, ensuring that the system meets the specifications and requirements documented in Chapter 3.

- **Functional Testing 1:** Login with Different Roles is presented in Table 6.4
- **Objective:** To ensure that the correct dashboard and navigation interface load according to the user's role.

Table 6.4 Login Functional Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Login as a Patient	Email: patient1@aidlink.com Password: Patient@123	Patient dashboard loads with patient-specific navigation	Pass
2	Login as a Doctor	Email: doctor1@aidlink.com Password: Doctor@123	Doctor dashboard loads with doctor-specific navigation	Pass
3	Login as an Admin	Email: admin@aidlink.com Password: Admin@123	Admin dashboard loads with admin-specific navigation	Pass

- **Functional Testing 2:** Appointment Booking Workflow is presented in Table 6.5
- **Objective:** To ensure that the complete appointment booking process functions correctly from doctor search to confirmation.

Table 6.5 Appointment Booking Functional Test Case

No.	**Test Case/Test Script**	**Attribute and Value**	**Expected Result**	**Result**
1	Search and select a doctor, then book an appointment	Specialization: Cardiologist	Appointment is created with status "Pending" and doctor receives notification	Pass
2	Doctor approves the pending appointment	Appointment ID: APT1023	Appointment status updates to "Confirmed" and patient receives notification	Pass

- **Functional Testing 3:** In-App Chat Functionality is presented in Table 6.6
- **Objective:** To ensure that messages are sent, encrypted, and delivered correctly between patients and doctors.

Table 6.6: In-App Chat Functional Test Case

No.	**Test Case/Test Script**	**Attribute and Value**	**Expected Result**	**Result**
1	Send a text message from patient to doctor	Message: "Is my appointment confirmed?"	Message is delivered instantly and visible in doctor's chat window	Pass
2	Attempt to send an empty message	Message: ""	System displays "Message cannot be empty"	Pass

6.1.4 Integration Testing

Integration testing was performed to verify that different modules of AidLink work correctly together as a complete system.

Table 6.7 presents the integration test cases covering the end-to-end patient appointment and prescription flow.

Table 6.7: Integration Test Case

No.	Test Case/Test Script	Attribute and Value	Expected Result	Result
1	Patient logs in, searches for a doctor, and books an appointment	Specialization: Dermatologist	Login successful, doctor search returns results, and appointment is booked with "Pending" status	Pass
2	Doctor logs in, views the new appointment, and confirms it	Appointment ID: APT1045	Appointment status updates to "Confirmed" and patient receives a real-time notification	Pass
3	Doctor issues a prescription after consultation	Appointment ID: APT1045	Prescription is generated as a PDF, uploaded to Cloud Storage, and becomes visible to the patient	Pass
4	Patient views and downloads the issued prescription	Patient ID: PAT2031	Prescription appears in the patient's prescription history with a working download link	Pass

6.2 Automated Testing

In addition to manual testing, automated testing was performed to validate core application logic and ensure consistent behavior across repeated test executions.

6.2.1 Tools Used

Table 6.8 presents the tools used for automated testing of the AidLink system.

Table 6.8: Tools Used

Tool Name	Tool Description	Applied On	Results
Flutter Test	A unit and widget testing framework built into the Flutter SDK, used to test individual widgets and functions in isolation	FR-001, FR-002, FR-007, FR-009	All test cases passed with expected widget rendering and function outputs
Firebase Test Lab	A cloud-based testing service used to run automated UI tests across multiple real and virtual Android and iOS devices	FR-005, FR-007, FR-010, NFR Portability	All tests passed across Android 8.0+ and iOS 13+ devices
Postman	An API testing tool used to validate Firebase Cloud Function endpoints and verify correct request and response handling	FR-011, FR-017, NFR Performance	All endpoint tests returned expected responses within 3 seconds
Mockito	A mocking framework used in combination with Flutter Test to simulate Firebase service responses during unit testing	FR-001, FR-007, FR-011	All mocked test cases passed without requiring live Firebase calls

7. Conclusion and Future Work

This chapter presents the overall conclusion of the AidLink project, summarizing the objectives achieved and the value delivered to patients and doctors, followed by a discussion of potential future enhancements that could extend the system's capabilities beyond its current scope.

7.1 Conclusion

AidLink was developed to address the lack of a unified, verified, and accessible digital healthcare platform in Pakistan, where patients struggle to locate credentialed doctors and maintain continuous communication and medical records outside of physical consultations. Through the development of three integrated interfaces, a Flutter mobile application for patients, a Flutter Web portal for doctors, and a Flutter Web portal for administrators, the project successfully delivers a complete healthcare ecosystem powered by a Firebase backend.

The system fulfills all twenty functional requirements identified during the requirement analysis phase, enabling patients to search for verified doctors using name, specialization, and location filters, book appointments with real-time slot availability, communicate through an end-to-end encrypted in-app chat, and access their complete prescription history at any time. Doctors are equipped with tools to manage their availability schedules, handle appointment requests, and issue digital prescriptions, while administrators maintain platform integrity through a structured doctor verification workflow, user account management, and review moderation.

The architecture of AidLink, built on Object-Oriented Design principles and developed using the Iterative and Incremental software development lifecycle, ensured that the system was constructed in manageable, testable phases, allowing continuous refinement based on evaluation at each milestone. The integration of Firebase Firestore, Cloud Storage, Cloud Functions, and Cloud Messaging, alongside the Google Maps API for location-based doctor discovery, demonstrates the practical application of modern cloud-based technologies in solving a real-world healthcare accessibility problem.

Testing and evaluation, covering unit testing, functional testing, integration testing, and automated testing, confirmed that the system performs reliably across all three interfaces and meets the non-functional requirements established for usability, performance, security, reliability, availability, maintainability, and portability. Overall, AidLink successfully achieves its core objective of bridging the gap between patients seeking healthcare and the verified medical professionals who can provide it, offering a scalable foundation for digital healthcare delivery in Pakistan

7.2 Future Work

While AidLink successfully delivers its core objectives, several enhancements could be incorporated in future iterations of the system to further expand its capabilities and reach.

Video consultation functionality could be integrated to allow patients to consult with doctors remotely in real time, reducing the need for physical visits for minor or follow-up consultations. This would be particularly valuable for patients in remote areas of Khyber Pakhtunkhwa and other underserved regions of Pakistan.

Online payment integration could be added to allow patients to pay consultation fees directly through the application using local payment gateways such as JazzCash or Easypaisa, streamlining the transaction process and reducing reliance on cash payments at clinics.

An AI-based doctor recommendation engine could be developed to analyze a patient's symptoms and medical history to suggest the most appropriate specialist, improving the accuracy and speed of the doctor discovery process beyond simple keyword-based search.

Pharmacy integration could be introduced to allow patients to order prescribed medicines directly through the platform, with delivery tracking, creating a more complete end-to-end healthcare experience.

Finally, expanding the administrator analytics module to include predictive insights, such as forecasting appointment demand by specialization or region, could help healthcare administrators make informed decisions about resource allocation and doctor onboarding priorities.

8. References

[1] C. L. Ventola, "Mobile devices and apps for health care professionals: Uses and benefits," *Pharmacy and Therapeutics*, vol. 39, no. 5, pp. 356–364, 2014.

[2] R. S. H. Istepanian, E. Jovanov, and Y. T. Zhang, "Guest editorial introduction to the special section on m-health: Beyond seamless mobility and global wireless health-care connectivity," *IEEE Transactions on Information Technology in Biomedicine*, vol. 8, no. 4, pp. 405–414, 2004.

[3] C. Free, G. Phillips, L. Galli, L. Watson, L. Felix, P. Edwards, V. Patel, and A. Haines, "The effectiveness of mobile-health technology-based health behaviour change or disease management interventions for health care consumers: A systematic review," *PLOS Medicine*, vol. 10, no. 1, p. e1001362, 2013.

[4] K. Abouelmehdi, A. Beni-Hssane, H. Khaloufi, and M. Saadi, "Big healthcare data: Preserving security and privacy," *Journal of Big Data*, vol. 5, no. 1, pp. 1–18, 2018.

[5] P. Nawrocki, B. Wrona, M. Marczak, and W. Sniezynski, "A comparison of native and cross-platform frameworks for mobile applications," *Computer*, vol. 54, no. 3, pp. 18–28, 2021.

[6] I. Sommerville, *Software Engineering*, 9th ed. Boston, MA, USA: Addison-Wesley, 2011.

[7] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2014

AidLink_100_-1

ORIGINALITY REPORT

17%	8%	3%	14%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Higher Education Commission Pakistan Student Paper	7%
2	Submitted to COMSATS University Islamabad – Abbottabad Campus Student Paper	1%
3	Submitted to Islington College,Nepal Student Paper	< 1%
4	Submitted to Universiti Teknologi Malaysia Student Paper	< 1%
5	Submitted to British University In Dubai Student Paper	< 1%
6	Submitted to Quaid-e-Azam University, Islamabad Student Paper	< 1%
7	dspace.lboro.ac.uk Internet Source	< 1%
8	v.vibdoc.com Internet Source	< 1%
9	Submitted to From HEC Pool Student Paper	< 1%
10	Submitted to University of Greenwich Student Paper	< 1%
11	dspace.daffodilvarsity.edu.bd:8080 Internet Source	< 1%
12	www.ijraset.com Internet Source	< 1%
13	Submitted to Universiti Tun Hussein Onn Malaysia Student Paper	< 1%
14	Renato M Toasa. ".NET MAUI vs. Cross-Platform Mobile Development Frameworks: A Comparative Analysis and State of the Art", International Journal of Engineering Insights, 2025 Publication	< 1%

15	ijcrt.org Internet Source	< 1 %
16	tau.edu.ng Internet Source	< 1 %
17	Submitted to Universiti Poly-Tech Malaysia Student Paper	< 1 %
18	indianjournalofcapitalmarkets.com Internet Source	< 1 %
19	Submitted to Ravensbourne Student Paper	< 1 %
20	vehari.comsats.edu.pk Internet Source	< 1 %
21	www.ijournalse.org Internet Source	< 1 %
22	C.E. Dickerson, Siyuan Ji. "Essential Architecture and Principles of Systems Engineering", CRC Press, 2021 Publication	< 1 %
23	Submitted to International Islamic University Student Paper	< 1 %
24	Submitted to UOW Malaysia KDU University College Sdn. Bhd Student Paper	< 1 %
25	Submitted to GIFT University Student Paper	< 1 %
26	Submitted to University of Maryland, Global Campus Student Paper	< 1 %
27	Submitted to University of Western Sydney Student Paper	< 1 %
28	scholar.ppu.edu Internet Source	< 1 %
29	Submitted to COMSATS Institute of Information Technology, Attock Student Paper	< 1 %
30	ejournal.unkhair.ac.id Internet Source	< 1 %
31	Submitted to Birzeit University Main Library Student Paper	< 1 %

32	Submitted to De Montfort University Student Paper	< 1 %
33	Submitted to Florida International University Student Paper	< 1 %
34	Submitted to RMIT University Student Paper	< 1 %
35	Submitted to University of Bolton Student Paper	< 1 %
36	Submitted to University of Keele Student Paper	< 1 %
37	Submitted to University of Wales Institute, Cardiff Student Paper	< 1 %
38	www.mass.gov Internet Source	< 1 %
39	Submitted to Colorado Technical University Online Student Paper	< 1 %
40	Submitted to Kaplan College Student Paper	< 1 %
41	Submitted to Nottingham Trent University Student Paper	< 1 %
42	Submitted to Swinburne University of Technology Student Paper	< 1 %
43	Submitted to Wawasan Open University Student Paper	< 1 %
44	ntnuopen.ntnu.no Internet Source	< 1 %
45	www.theijire.com Internet Source	< 1 %
46	Submitted to Ohio University Student Paper	< 1 %
47	downloads.hindawi.com Internet Source	< 1 %
48	energycapinc.applytojob.com Internet Source	< 1 %

49	Internet Source	< 1 %
50	rps.wku.edu.et Internet Source	< 1 %
51	www.dremio.com Internet Source	< 1 %
52	Richard C. Fries. " Medical Device Design", CRC Press, 2019 Publication	< 1 %
53	acseusa.org Internet Source	< 1 %
54	fdocuments.us Internet Source	< 1 %
55	repository.learn-portal.org Internet Source	< 1 %
56	wiki.onap.org Internet Source	< 1 %
57	www.bis.org Internet Source	< 1 %
58	www.coursehero.com Internet Source	< 1 %
59	www.freelancer.in Internet Source	< 1 %
60	www.programcreek.com Internet Source	< 1 %
61	www.theseus.fi Internet Source	< 1 %
62	Thomas B. Hilburn, Massood Towhidnejad. "Software Engineering Practice – A Case Study Approach", CRC Press, 2020 Publication	< 1 %

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		

Aidlink_100_-1 Aidlink_100_-1

AidLink_100_-1

Quick Submit

Quick Submit

COMSATS University Islamabad - Abbottabad Campus

Document Details

Submission ID

trn:oid:::1:3623898166

Submission Date

Aug 7, 2026, 3:09 PM GMT+5

Download Date

Aug 7, 2026, 3:16 PM GMT+5

File Name

AidLink_100_-1.docx

File Size

4.6 MB

68 Pages

12,290 Words

77,260 Characters

turnitin Page 1 of 70 - Cover Page Submission ID trn:oid:::1:3623898166

6% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

17 AI-generated 6%
Likely generated or likely generated and revised by AI.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

COMSATS University Islamabad Abbottabad Campus
Department of Computer Science

Guidelines for Thesis Formatting

S. No.	Guidelines	Checked by Supervisor Clear/Not Clear	Checked by Committee Clear/Not Clear
1	Normal Text Font: 12 and Times New Roman	✓	
2	Line spacing between normal text is 1.15	✓	
3	References (If required)	✓	
4	References calling in text	✓	
5	Caption Format (size: 11, font: Italic)	✓	
6	Figure numbering & caption (below the figure).	✓	
7	Figure numbering calling in text	✓	
8	Equation number	✓	
9	Equation numbering calling in text	✓	
10	Table numbering & caption (on top of the table).	✓	
11	Table calling in text	✓	
12	Ctrl+F and check for double spaces	✓	
13	Figures on top of text	✓	
14	Line spaces between figures and text	✓	
15	Line spaces between equations and text	✓	
16	Figure and Tables should lie inside the page margin.	✓	
17	Figure should be clearly visible.	✓	
18	Equations formatting should be consistent	✓	
19	No bullet list, proper sentences, only used bullets when needed.	✓	
20	No unnecessary capitalization.	✓	
21	Consistent heading size and bullets.	✓	
22	Header & Footer starts from Chapter # 1. (size: 11 & Italic)	✓	
23	"Project Title" in header & "Department of Computer Science" in footer	✓	
24	Layout Margin should be Normal	✓	
25	Heading numbering up to 3rd level. Use bullets for 4th and onwards.	✓	
26	At least one sentence explanation after every heading.	✓	
27	Format of title pages heading is: size 14, underline, bold, all caps & center align such as TABLE OF CONTENTS etc.	✓	
28	Format of chapter name heading is: size: 18 & bold	✓	
29	Heading 1 size: 14 & bold. H2 size: 13 & bold. H3 size:12 & bold.	✓	
30	Proper indent should be used with heading and text.	✓	

Note: These guidelines must be attached at the end of thesis:

Supervisor Sign Committee Sign

DEPARTMENT OF COMPUTER SCIENCE,
COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS

FINAL APPROVAL

This is to certify that we have read the thesis submitted by **Naveed Alam** having registration number **CIIT/FA22-BCS-089/ATD** in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Computer Science.**

1. External Examiner _____

Dr. Muhammad Fayyaz

Associate Professor

FAST Faisalabad

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Sadaf Riaz

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26

DEPARTMENT OF COMPUTER SCIENCE,

COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS

FINAL APPROVAL

This is to certify that we have read the thesis submitted by **Moazzam Ali** having registration number **CIIT/FA22-BCS-102/ATD** in the session **SP26**. It is our judgment that this thesis is of sufficient standard to warrant its acceptance by the COMSATS University Islamabad, Abbottabad Campus for the **Bachelor of Computer Science.**

1. External Examiner _____

Dr. Muhammad Fayyaz

Associate Professor

FAST Faisalabad

2. Internal Examiner _____

Dr. Syed Faraz Ahmed

3. Supervisor _____

Sadaf Riaz

4. Head of Department _____

Dr. Zia ur Rehman

Department of Computer Science,

COMSATS UNIVERSITY ISLAMABAD,

Abbottabad Campus.

Dated: 11/08/26